

Capstone Project Final Report

Submitted to

Faculty of Engineering and Natural Sciences

Department of Computer Engineering

In partial fulfillment of the requirements for the degree

BACHELOR of SCIENCE in ISTİNYE UNIVERSITY

*TinyML – Enabled Indoor Pet Tracking and Anomaly
Detection Using BLE RSSI: An Edge – Centric
Approach*

Ahsen DURSUN 220901599

Ebru ÇEPNİ 210901002

Süreyya YILDIRIM 21090137

Supervisor: Hüsamettim OSMANOĞLU

Istanbul

May 2026

STUDENT DECLARATION

By submitting this report, as partial fulfillment of the requirements of the Capstone Project, the students promise on penalty of failure of the course that

- They have given credit to and declared (by citation), any work that is not their own (e.g. parts of the report that is copied/pasted from the Internet, design or construction performed by another person, etc.),
- They have not received unpermitted aid for the project design, construction, report or presentation,
- They have not falsely assigned credit for work to another student in the group, and not take credit for work done by another student in the group.

Abstract

Indoor pet tracking in smart home environments requires a monitoring system that can estimate location and interpret basic behavioral patterns with low latency, low cost, and minimal dependence on external infrastructure. However, indoor tracking is challenging because Global Positioning System (GPS) signals are not suitable for enclosed spaces, while Bluetooth Low Energy (BLE) Received Signal Strength Indicator (RSSI) measurements are highly sensitive to multipath propagation, obstacles, signal reflection, and environmental noise. In addition, cloud-based tracking architectures may introduce latency, communication overhead, and privacy concerns. This study proposes a lightweight, edge-based indoor pet tracking and behavior monitoring framework implemented on ESP32 hardware. The system uses a BLE beacon attached to the pet and three ESP32-based receiver nodes deployed in different indoor locations. The collected BLE RSSI measurements are transmitted to a master ESP32 node and combined into an RSSI fingerprint vector consisting of living room, kitchen, and bedroom RSSI features. A lightweight Multilayer Perceptron (MLP) model is trained and deployed using TensorFlow Lite Micro to perform real-time, on-device inference for indoor state estimation. To address RSSI instability, lightweight signal validation and majority voting mechanisms are integrated into the inference pipeline. Beyond room-level localization, the framework also incorporates feeding area detection as a dedicated behavioral micro-zone and stationary behavior analysis using temporal rules. Experimental results show that the MLP model achieves 91.73% average accuracy under 5-fold cross-validation, 96.74% accuracy under session-based split evaluation, and 80.1% overall accuracy in the realistic dynamic scenario. Overall, the proposed framework demonstrates that BLE RSSI fingerprinting and TinyML-based edge intelligence can support practical, low-cost, and privacy-aware indoor pet tracking under constrained hardware conditions.

Keywords: TinyML; indoor localization; BLE RSSI fingerprinting; ESP32; ESP-NOW; edge computing; behavior monitoring; indoor pet tracking.

Table of Contents

1	INTRODUCTION.....	1
2	LITERATURE REVIEW.....	5
2.1	<i>THEORETICAL BACKROUND.....</i>	5
2.1.1	Fundamental Principles of BLE and RSSI in Indoor Environments	5
2.1.2	Localization Techniques: Fingerprinting, Triangulation, and GPS.....	7
2.1.2.1	Global Positioning System (GPS).....	7
2.1.2.2	Triangulation and Trilateration	7
2.1.2.3	Fingerprinting (Scene Analysis)	7
2.1.3	TinyML and Model Optimization for Resource- Constrained Devices	9
2.1.3.1	Quantization and Pruning Methodologies.....	10
2.1.3.2	Benefits of On-Device Inference.	10
2.1.4	Time-Series Anomaly Detection and Behavioral Analytics.....	11
2.1.4.1	Stationary Detection (Locomotor Activity Analysis)	11
2.1.4.2	Proximity Analytics and Behavioral Budgeting	12
2.2	<i>REVIW OF EXISTING APPROACHES</i>	13
2.2.1	Edge–Cloud Computing for IoT Intelligence	13
2.2.2	TinyML-Based Learning Approaches	14
2.2.3	RSSI and BLE-Based Indoor Localization Techniques	16
2.2.4	Comparative Evaluation and Selection of Localization Methodology	17
2.3	<i>CRITICAL ANALYSIS.....</i>	19
2.3.1	Strengths and Weaknesses of Methods.....	19
2.3.2	Divergent Approaches in Architecture and Data.....	20
2.3.3	Identified Research Gaps and Project Motivation	20
2.4	<i>SUMMARY AND CONNECTION TO THE PROPOSED PROJECT</i>	21
3	METHODOLOGY.....	23
3.1	<i>SYSTEM OVERVIEW</i>	23
3.2	<i>HARDWARE INFRASTRUCTURE</i>	24
3.2.1	ESP32 and ESP-NOW Communication	24
3.2.2	BLE Technology and BLE Beacon	25
3.3	<i>DATA COLLECTION PROCESS.....</i>	26
3.4	<i>DATA PREPROCESSING.....</i>	29
3.5	<i>LOCALIZATION MODEL.....</i>	29
3.5.1	Fingerprinting Rationale	30

3.5.2	MLP Rationale.....	30
3.6	<i>FEEDING AREA DETECTION STRATEGY</i>	32
3.6.1	Rule-Based Feeding Area Detection.....	32
3.6.2	Machine Learning-Based Feeding Area Detection.....	33
3.7	<i>PROXIMITY ANALYTICS AND BEHAVIORAL BUDGETING</i>	34
3.8	<i>TINYML DEPLOYMENT</i>	36
3.9	<i>EVALUATION STRATEGY AND PERFORMANCE METRICS</i>	37
4	RESULTS	40
5	DISCUSSION AND CONCLUSION	46
5.1	<i>DISCUSSION OF LOCALIZATION PERFORMANCE</i>	46
5.2	<i>DISCUSSION OF VALIDATION STRATEGIES</i>	47
5.3	<i>DISCUSSION OF TINYML DEPLOYMENT</i>	48
5.4	<i>DISCUSSION OF FEEDING AREA AND BEHAVIORAL MONITORING</i>	49
5.5	<i>COMPARISON WITH EXISTING LITERATURE</i>	50
5.6	<i>LIMITATIONS</i>	51
5.7	<i>CONCLUSION</i>	52
	ACKNOWLEDGEMENT	54
	APPENDIX	54
	REFERENCES	54

List of Tables

Table 1: Comparison of GPS, triangulation, and fingerprinting-based localization techniques	8
Table 2: Architectural comparison of cloud, edge, and tinyML approaches	16
Table 3: Comparative summary of related literature	19
Table 4: The stationary data collection sessions	27
Table 5: The dynamic data collection session.....	28
Table 6: Feeding Area Threshold Analysis.....	33
Table 7: Classification performance metrics of the selected MLP model under dataset C.....	42

List of Figures

Figure 1: Comparison of indoor localization paradigms.....	8
Figure 2: General system overview and applied house map.....	23
Figure 3: ESP-NOW communication architecture.....	24
Figure 4: Custom-configured BLE beacon	26
Figure 5: RSSI fingerprint-based localization pipeline and lightweight MLP architecture....	31
Figure 6: TinyML model deployment pipeline	37
Figure 7: Accuracy comparison of evaluated machine learning models	40
Figure 8: Confusion matrix of the selected MLP model under dataset C.....	42
Figure 9: Per-class recall comparison of MLP, KNN, random forest, decision tree, and SVM models under datasets.....	42
Figure 10: Feeding area detection method comparison under dataset B with random sampling	43
Figure 11: Feeding area recall performance compasion under dataset B and dataset C	43
Figure 12: Localization accuracy results across all datasets obtained from experimental te...	43
Figure 13: Real-time feeding interaction timeline over a 4-hour observation period.....	44
Figure 14: Cumulative feeding duration during a 4-hour observation period.....	44

List of Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
dB	Decibel
dBm	Decibel-Milliwatts
DL	Deep Learning
ESP	Espressif Systems Platform
ESP32	Espressif Systems 32-bit Microcontroller
ESP-NOW	Espressif Now
F1-score	F-measure / Harmonic Mean of Precision and Recall
FN	False Negative
FP	False Positive
GPIO	General Purpose Input/Output
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
IPS	Indoor Positioning System
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LoS	Line-of-Sight
MAE	Mean Absolute Error
MCU	Microcontroller Unit
ML	Machine Learning
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NLoS	Non-Line-of-Sight
NN	Neural Network

PoI	Point of Interest
QoS	Quality of Service
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RF	Radio Frequency
RMSE	Root Mean Squared Error
ROM	Read-Only Memory
RSSI	Received Signal Strength Indicator
SDK	Software Development Kit
SRAM	Static Random-Access Memory
SVM	Support Vector Machine
TCP/IP	Transmission Control Protocol / Internet Protocol
TFLite	TensorFlow Lite
TFLite Micro	TensorFlow Lite for Microcontrollers
TFLM	TensorFlow Lite for Microcontrollers
TinyML	Tiny Machine Learning
TN	True Negative
TP	True Positive
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
USB	Universal Serial Bus
VLC	Visible Light Communication
Wi-Fi	Wireless Fidelity

Section 1:

Introduction

1 INTRODUCTION

With the rapid expansion of the Internet of Things (IoT), the amount of data generated by connected devices has increased significantly, creating new challenges for traditional cloud-centric computing architectures. Although cloud computing provides high computational capacity and centralized data processing, it also introduces limitations such as transmission latency, bandwidth consumption, and dependence on continuous internet connectivity. These limitations become more critical in real-time monitoring applications, where data must be processed quickly and reliably. Therefore, processing data closer to its source has become an important design requirement, making edge computing a practical alternative for latency-sensitive and resource-aware IoT systems [1].

In this context, Tiny Machine Learning (TinyML) extends the edge computing paradigm by enabling machine learning inference directly on resource-constrained microcontroller units. Instead of transmitting raw sensor data to external servers for processing, TinyML-based systems allow data to be analyzed locally on embedded devices with limited memory, computation, and power capacity. This on-device intelligence reduces communication overhead, lowers end-to-end latency, and supports privacy-preserving operation by keeping raw data closer to its source. As a result, TinyML provides a suitable foundation for real-time IoT applications that require autonomous decision-making under strict hardware constraints [1], [2]. Indoor localization represents one of the application areas where edge-based and low-power intelligent systems can provide significant benefits. Since the Global Positioning System (GPS) is unreliable in enclosed environments, alternative wireless technologies are required for indoor positioning tasks. Bluetooth Low Energy (BLE) has become a widely adopted option due to its low power consumption, cost efficiency, and compatibility with consumer-grade IoT devices [3], [4]. However, BLE-based localization commonly relies on Received Signal Strength Indicator (RSSI) measurements, which are highly sensitive to indoor environmental conditions such as multipath propagation, signal reflection, obstacles, and noise. These fluctuations make direct distance estimation unreliable and motivate the use of data-driven approaches such as RSSI fingerprinting, where location-specific signal patterns are learned instead of explicitly calculating geometric distance [5].

Although existing studies demonstrate the potential of BLE and TinyML-based approaches for indoor tracking, several limitations remain for small-scale home monitoring scenarios. Many indoor tracking systems primarily focus on estimating location, while behavioral interpretation, such as feeding-related activity or prolonged inactivity, is often treated as a secondary concern.

In addition, some practical pet tracking implementations still rely on centralized processing infrastructures, which may introduce latency, connectivity dependence, and additional system complexity [6]. Recent TinyML-based indoor tracking studies show that lightweight models can be deployed for localization tasks on constrained devices; however, integrating room-level localization with behavior-aware monitoring on an ESP32-based edge architecture remains a relevant challenge [7]. Therefore, there is a need for an indoor pet tracking system that can perform local RSSI-based localization while also supporting basic behavioral analysis directly at the edge.

In this study, an edge-centric indoor pet tracking system is developed using a BLE beacon attached to the pet and three ESP32-based receiver nodes deployed in different indoor locations. The receiver nodes collect RSSI measurements from the BLE beacon, and the master ESP32 combines these measurements into an RSSI fingerprint vector composed of three features: *rssi_living*, *rssi_kitchen*, and *rssi_bedroom*. This fingerprint is processed by a lightweight Multilayer Perceptron (MLP) model to estimate the pet's indoor state, including *living_room*, *kitchen*, *bedroom*, and *feeding_area*. Beyond room-level localization, the system also incorporates feeding area detection as a dedicated behavioral micro-zone and stationary behavior analysis using temporal rules to support basic behavior-aware monitoring. To mitigate spatial and temporal RSSI anomalies, lightweight signal validation and majority voting mechanisms are integrated directly into the inference pipeline. By deploying the localization model on the master ESP32 using TinyML, the proposed system performs inference locally and reduces dependence on external servers, making it suitable for low-latency and resource-constrained indoor IoT applications [2], [8].

The main contribution of this study is the design and implementation of a lightweight, edge-based indoor pet tracking framework that combines BLE RSSI fingerprinting, TinyML-based localization, and behavior-aware monitoring on ESP32 hardware. Unlike cloud-dependent tracking systems, the proposed architecture performs inference directly on the master ESP32, enabling local decision-making and reducing communication overhead. The system is evaluated using stationary and dynamic RSSI datasets, different machine learning models, multiple validation strategies, and feeding area detection experiments. The remainder of this thesis is organized as follows: the Literature Review section discusses the theoretical background and related studies on edge computing, TinyML, BLE/RSSI-based indoor localization, and anomaly-aware monitoring; the Methodology section explains the proposed hardware architecture, data collection process, model development, and TinyML deployment;

the Results section presents the experimental findings; and the final section discusses the conclusions, limitations, and possible future improvements of the system.

Section 2:

Literature Review

2 LITERATURE REVIEW

This section reviews the technical foundations and existing approaches that inform the proposed system. Within this scope, the literature is examined through four main perspectives: edge and cloud computing for IoT intelligence, TinyML-based learning on resource-constrained devices, BLE/RSSI-based indoor localization, and time-series-oriented anomaly and behavioral analysis. First, edge computing is discussed as an alternative to cloud-centric IoT architectures, particularly for applications that require low latency, reduced communication overhead, and local decision-making [1]. Then, TinyML is reviewed as a practical approach for deploying lightweight machine learning models on microcontroller units under strict memory, computation, and power constraints [2], [8]. After that, BLE and RSSI fingerprinting-based indoor localization methods are analyzed with respect to signal instability, multipath propagation, and data-driven position estimation [3], [5]. Finally, anomaly-aware monitoring and behavioral interpretation are reviewed to connect localization outputs with higher-level pet activity analysis. Through this organization, the literature review establishes the technical background, identifies existing limitations, and clarifies the motivation for the proposed edge-centric indoor pet tracking system.

2.1 THEORETICAL BACKGROUND

2.1.1 Fundamental Principles of BLE and RSSI in Indoor Environments

BLE has become a cornerstone technology for Indoor Positioning Systems (IPS) due to its low power consumption, cost efficiency, and widespread availability in consumer-grade devices [4]. In recent years, BLE-based indoor localization has attracted increasing attention, particularly in smart environments such as homes, offices, and healthcare facilities, where energy-efficient and scalable positioning solutions are required [9]. The necessity for such systems stems from the inherent limitations of the GPS in enclosed environments. While GPS is widely used for outdoor localization, it is unsuitable for indoor scenarios because it relies on a clear line-of-sight (LoS) between satellites and receivers, typically providing limited accuracy and high energy consumption that is insufficient for most indoor applications [10]. These constraints have motivated the exploration of alternative sensing modalities, with radio frequency (RF)-based localization methods emerging as the most widely adopted due to their high adaptability and reliance on existing wireless infrastructure [5].

Within the RF category, BLE has emerged as a particularly attractive solution, offering fine-grained control over transmission power and advertising intervals while maintaining ultra-low energy consumption [4]. To leverage these RF signals effectively, researchers have developed various positioning methodologies, among which fingerprint-based localization stands out as one of the most robust approaches. Unlike distance-based techniques that estimate position by modeling signal propagation through rigid analytical formulas, fingerprinting is a data-driven method that compares real-time signal measurements with a pre-recorded database of signal characteristics [5]. This approach is especially well-suited for complex indoor environments where multipath propagation, signal reflections, and non-line-of-sight (NLoS) conditions significantly degrade the accuracy of standard path-loss models.

A commonly used analytical model for RSSI-based ranging is the log-distance path-loss model, which can be expressed as in Equation (1).

$$RSSI(d) = RSSI(d_0) - 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (1)$$

In the Equation (1), $RSSI(d_0)$ denotes the received signal strength at a reference distance d_0 , n is the path-loss exponent, and X_σ represents stochastic variation caused by shadowing, multipath fading, and environmental noise. Although this model suggests a logarithmic relationship between signal strength and distance, the parameters n and X_σ vary significantly in indoor environments. As a result, direct distance estimation from RSSI becomes unreliable in practice, which further motivates the adoption of data-driven approaches such as fingerprinting instead of purely model-based localization.

The practical implementation of RSSI fingerprinting is typically divided into two distinct phases: an offline phase and an online phase [5]. During the offline phase, RSSI measurements are collected at predefined reference points and stored in a database to train a machine learning model that maps signal patterns to spatial coordinates. In the online phase, real-time RSSI values from an unknown location are fed into this trained model, which performs pattern matching to estimate the most probable position. While the effectiveness of this process depends on factors such as the density of reference points and the quality of the fingerprint database, it is widely recognized as a highly accurate and cost-effective technique-particularly when combined with machine learning-based inference [5], [11].

Despite its robustness, the inherent instability of RSSI measurements over time due to environmental dynamics remains a challenge, often requiring periodic updates to the fingerprint

database to maintain reliable performance [5]. Nevertheless, the flexibility of fingerprinting has allowed its principles to extend beyond BLE to other modalities, such as Visible Light Communication (VLC), reinforcing its status as a dominant solution for indoor positioning systems. Overall, the adaptability of RF-based methods and the robustness of fingerprinting approaches provide a natural pathway toward integrating lightweight machine learning and TinyML techniques to enable efficient, on-device localization in resource-constrained environments.

2.1.2 Localization Techniques: Fingerprinting, Triangulation, and GPS

In the field of modern localization, position estimation is addressed through distinct methodologies that cater to different environmental constraints. The most prominent paradigms include satellite-based systems, geometric algorithms, and database-dependent signal mapping.

2.1.2.1 Global Positioning System (GPS)

GPS is the global standard for outdoor positioning, enabling 'geofencing' in wide open areas using satellite signals. The system operates by measuring the time delay between satellite transmissions to calculate a receiver's coordinates. However, GPS often reaches its limitations in indoor environments where building materials create signal blind spots, preventing the receiver from maintaining a line-of-sight with the satellites [12].

2.1.2.2 Triangulation and Trilateration

In GPS-deprived environments, the triangulation algorithm provides an alternative by determining a position based on geometric principles. This method requires a minimum of three access points or anchors regardless of the measurement area's geometry [13]. The process relies on the geometric combination of distance or angle information from these reference points. Conceptually, this is represented as the intersection of three distinct circles, where the overlapping point identifies the target's coordinates [14]. The accuracy of this geometric approach is influenced by the strategic placement of sensors, where maintaining an angle close to 90 degrees between the sensors and the target narrows the 'uncertainty ellipses' [15].

2.1.2.3 Fingerprinting (Scene Analysis)

Fingerprinting is a data-driven localization paradigm that operates in two distinct phases: offline (calibration) and online (matching) [16], [17]. During the offline phase, the environment is

surveyed to create a 'radio map,' where RSSI (Received Signal Strength Indicator) samples are collected and stored as digital signatures for specific coordinates [18]. In the online phase, the real-time RSSI vector received from the target is compared against this pre-recorded database using algorithms—often involving machine learning or K-Nearest Neighbor (KNN).

Figure 1 illustrates the main indoor localization paradigms and summarizes their operational characteristics and suitability for indoor monitoring environments.

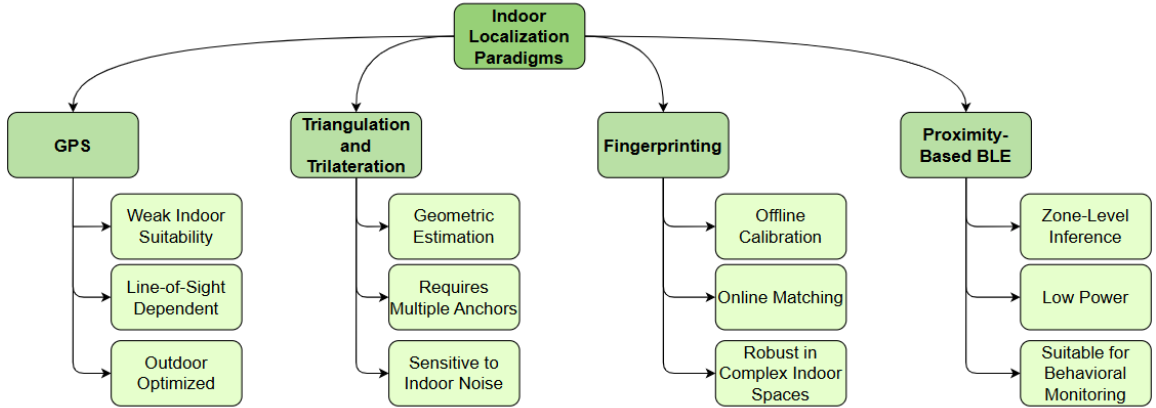


Figure 1: Comparison of indoor localization paradigms

Table 1 summarizes the fundamental characteristics of GPS, triangulation, and fingerprinting-based localization approaches discussed in Section 2.2.2.

Table 1: Comparison of GPS, triangulation, and fingerprinting-based localization techniques

Feature	GPS	Triangulation	Fingerprinting
Method	Global Positioning System	Geometric Intersection	Scene Analysis
Core Principle	Satellite signal trilateration & time-delay [12].	Geometric intersection of angles/distances from 3+ anchors [13], [14].	Offline radio maps & online matching [16], [17].
Indoor Suitability	Low (Signal attenuation by walls).	Moderate (Sensitive to obstacles).	High (Ideal for complex layouts).
Infrastructure	Global satellite network; no local setup.	Requires 3+ anchors with specific 90° placement [15].	Standard BLE beacons; requires offline calibration.
Strength	High accuracy in open areas; no local maintenance.	Mathematical simplicity; no prior database needed [19].	Captures unique environmental signal patterns [18].
Weakness	High power consumption; "blind spots" in buildings [12].	High error rate due to signal shadowing and multipath [14]	Time-consuming initial setup (radio mapping).
Relevance to Project	Not Selected: Ineffective for monitoring pets inside a house.	Not Selected: Household furniture creates too much geometric noise.	Selected: Most robust for TinyML-based tracking in dynamic homes.

2.1.3 TinyML and Model Optimization for Resource- Constrained Devices

While conventional machine learning models typically rely on cloud-based infrastructures with high computational capacity, recent studies show that inference can also be performed directly on resource-constrained Microcontroller Units (MCUs) using TinyML frameworks [2]. This shift is driven by the need to reduce latency, energy consumption, and communication overhead in edge-oriented IoT applications such as sensing, localization, and anomaly detection.

In this context, the ability to execute machine learning models on microcontroller-class devices is supported not only by model-level optimizations, but also by the development of lightweight deployment frameworks and hardware-oriented model design strategies. Embedded inference frameworks such as TensorFlow Lite for Microcontrollers (TFLM) provide a minimal runtime environment tailored for resource-constrained systems, enabling efficient on-device execution through static memory allocation, and reduced computational overhead [20].

In parallel, recent studies have explored architecture-level approaches that aim to design neural networks inherently suitable for such constrained environments. For example, MicroNets demonstrate that neural network structures can be optimized to minimize parameter count and operation complexity while preserving performance, making them more compatible with microcontroller-based platforms such as ESP32 or Arduino boards built on ARM Cortex-M architectures [21].

These developments indicate that TinyML is not limited to post-training optimization, but instead involves a broader design space in which model architecture, deployment framework, and hardware characteristics are jointly considered. Within this design space, techniques such as quantization-aware training further enhance deployment efficiency by adapting models to low-precision arithmetic during training, particularly for processors without floating-point support [22].

Despite these advances, running learning-based models on MCUs still imposes strict limitations on memory, computation, and power, which makes hardware-aware model optimization a key requirement [2], [8].

Consequently, several works highlight that the success of TinyML deployments depends not only on model architecture selection but also on the application of aggressive optimization techniques tailored to embedded hardware [8], [23]. In particular, lightweight neural networks such as shallow Convolutional Neural Networks (CNNs), Multilayer Perceptrons (MLPs), and compact classifiers are commonly adopted for signal-based tasks, including RSSI- driven localization and sensor anomaly detection, due to their favorable trade-off between

representational capacity and computational cost [8], [24]. These characteristics make such models particularly suitable for embedded signal-processing tasks, where structured input patterns such as RSSI measurements can be effectively modeled using low-complexity architectures.

2.1.3.1 Quantization and Pruning Methodologies

Among the most widely studied optimization techniques in TinyML literature are quantization and pruning. Quantization reduces the numerical precision of model parameters and activations, typically converting 32-bit floating-point (float32) representations into 8-bit integer (int8) formats. Empirical studies demonstrate that post-training and quantization-aware training approaches can reduce model memory usage by up to 75% while significantly accelerating inference on MCUs that lack dedicated floating-point units [23], [25]. Importantly, multiple works report that for classification and pattern-recognition tasks—such as RSSI fingerprinting and environmental sensing—the resulting accuracy degradation is often marginal when quantization is carefully applied [23].

Pruning complements quantization by removing redundant or low-importance neural connections that contribute minimally to model output. By inducing sparsity in the network architecture, pruning reduces both memory access operations and computational load, which are critical bottlenecks in ultra-low-power devices [8]. Recent studies show that structured pruning techniques can preserve model accuracy while enabling efficient execution within the kilobyte-level SRAM and flash memory budgets of commonly used MCUs [8], [24]. For localization and anomaly detection tasks, where input features often exhibit high correlation, pruning has been shown to be particularly effective in maintaining performance while reducing inference cost. The literature consistently frames these optimization techniques as a trade-off between accuracy and resource efficiency. While aggressive compression can lead to performance degradation, most TinyML-oriented studies conclude that moderate quantization and pruning achieve a favorable balance, enabling reliable inference under strict hardware constraints without sacrificing application-level usability [8], [23].

2.1.3.2 Benefits of On-Device Inference.

Performing inference directly on edge devices offers several advantages beyond computational feasibility. Prior studies highlight that on-device inference significantly reduces the energy overhead associated with continuous data transmission to cloud servers, often referred to as the “energy tax” of cloud-centric systems [2], [25]. This benefit is particularly important for

battery-powered IoT devices operating in indoor environments. In addition, local inference lowers end-to-end latency, enabling near real-time responsiveness in applications involving mobile entities such as indoor tracking. Processing data locally also enhances privacy, as raw sensor measurements remain on the device and only high-level information is transmitted when necessary [2]. Consequently, TinyML-based on-device inference provides a practical foundation for efficient, low-latency, and privacy-aware RSSI-based indoor localization systems.

Taken together, these findings demonstrate that TinyML enables a practical and efficient approach for deploying machine learning models directly on resource-constrained devices by combining lightweight architectures, hardware-aware optimization techniques, and embedded inference frameworks. While these design choices establish the technical feasibility of on-device intelligence, existing studies adopt different strategies in terms of model selection, optimization methods, and hardware platforms. Therefore, a closer examination of how these approaches are implemented in practice is necessary to better understand their trade-offs and applicability to real-world scenarios.

2.1.4 Time-Series Anomaly Detection and Behavioral Analytics

In the proposed system, localization data is processed not merely as static coordinates but as a continuous temporal stream of RSSI measurements. Time-series analysis allows for the extraction of meaningful behavioral patterns and the identification of deviations that signify physiological or environmental distress.

2.1.4.1 Stationary Detection (Locomotor Activity Analysis)

The proposed system models locomotor activity through the analysis of RSSI measurements represented as temporal signal sequences. Rather than relying on instantaneous signal values, stationary detection is formulated as a time-series anomaly detection problem, where deviations are assessed relative to historical behavioral patterns. This approach aligns with contextual and collective anomaly definitions commonly adopted in machine learning-based anomaly detection frameworks, where abnormality emerges from sustained deviations over time rather than isolated observations [1], [2].

Indoor wireless signal propagation is inherently affected by multipath fading, shadowing, and dynamic environmental conditions, resulting in highly variable RSSI measurements [5], [26].

To mitigate the impact of short-term fluctuations, the system evaluates RSSI variance within sliding temporal windows.

The variance of the RSSI sequence within a sliding window W can be defined as in Equation (2).

$$Var(W) = \frac{1}{N} \sum_{i=1}^N (r_i - \bar{r})^2 \quad (2)$$

In Equation (2), r_i denotes the RSSI samples in the window and $\bar{r}$ is their mean. When the variance remains below a predefined threshold τ for consecutive windows, the observed behavior is interpreted as a stationary period, as defined in Equation (3).

$$Var(W) < \tau \Rightarrow \text{stationary state} \quad (3)$$

This formulation enables the system to suppress transient fluctuations while capturing sustained inactivity patterns.

A sustained reduction in signal variance below an empirically determined threshold is interpreted as a stationary phase, indicating limited locomotor activity. Window-based segmentation has been shown to be effective for capturing motion-related signal characteristics in indoor IoT environments, particularly when combined with lightweight signal processing techniques suitable for edge deployment [16], [27].

Prolonged stationary periods are treated as collective anomalies rather than point anomalies, reflecting deviations from the subject's established movement routine. By comparing current stasis durations against historical activity baselines, the system differentiates expected resting behavior from atypical inactivity that may indicate abnormal conditions. This formulation improves robustness against transient noise while maintaining sensitivity to meaningful behavioral changes, consistent with embedded and real-time anomaly detection approaches proposed for resource-constrained IoT devices [27], [28].

2.1.4.2 Proximity Analytics and Behavioral Budgeting

Indoor positioning consists of two phases: In the offline (training) phase, unique "radio maps" are generated for each area of the house using RSSI values collected from ESP32 nodes. In the online (estimation) phase, real-time RSSI vectors received from all ESP32s are processed by a TinyML model running on an edge unit (Master ESP32). BLE-based proximity analysis is

considered suitable for continuous edge-based operation, as it provides sufficient spatial resolution for indoor behavior monitoring while maintaining low energy consumption [16], [29].

Continuous monitoring of RSSI patterns between the fixed nodes and the mobile tag allows the system to identify recurring proximity events associated with routine activities. These events are aggregated over long periods to create a behavioral budget representing the typical temporal distribution and duration of interactions with each Point of Interest (PoI). Unlike instantaneous indoor positioning approaches, this strategy focuses on long-term behavioral consistency, which has been shown to increase robustness in behavior classification tasks based on BLE signal processing and machine learning [29].

Formally, a behavioral budget can be represented as a set of temporal interaction features extracted for each Point of Interest (PoI), such as visit frequency f_k , cumulative dwell time T_k , and average visit duration D_k over a given observation interval. By comparing these features with historical baselines, the system can identify deviations in routine activity patterns without relying solely on instantaneous location estimates.

Anomaly detection is performed by comparing real-time proximity patterns with historical behavior profiles, utilizing pattern recognition principles commonly adopted in IoT-based anomaly detection systems [17], [30]. Significant deviations, such as a decrease in interaction frequency, irregular dwell times, or abnormal temporal distributions, are flagged as contextual anomalies rather than isolated events. Leveraging previous findings on RSSI-based indoor positioning and BLE performance optimization [5], [26], the system distinguishes purposeful interactions from incidental proximity caused by transient movement. Thus, behavior inference is based on meaningful activity patterns and remains resilient to the inherent noise of indoor wireless environments.

2.2 REVIEW OF EXISTING APPROACHES

2.2.1 Edge–Cloud Computing for IoT Intelligence

The rapid growth of IoT systems has increased the need to distribute computation between cloud and edge infrastructures to balance scalability, latency, bandwidth usage, and privacy. While cloud computing remains widely adopted due to its centralized management and elastic resources, cloud-centric architectures can become inefficient for latency-sensitive and data-intensive IoT applications because of continuous raw data transmission and network congestion. Edge computing addresses these limitations by moving parts of the processing

pipeline closer to the data source, enabling faster responses, and reducing communication overhead. Recent studies further highlight hybrid paradigms, such as fog computing, as effective compromises that combine cloud scalability with edge-level responsiveness [1].

Domain-focused research reinforces the importance of layered computing models. In smart agriculture, for example, surveys of cloud–fog–edge architectures show that although many systems still rely heavily on cloud processing, multi-layer designs are increasingly recommended to cope with connectivity limitations, distributed sensing, and real-time requirements. These studies also identify ongoing challenges -including energy consumption, privacy and security risks, hardware cost, and unreliable internet access-which limit the effectiveness of purely cloud-based solutions [18].

Beyond conceptual comparisons, recent project-level implementations demonstrate how distributed intelligence can be realized in practice. Cooperative architectures that integrate TinyML at the device level with edge-based decision support systems allow resource-constrained IoT devices to perform local inference and transmit only compact results to the edge for aggregation. This approach reduces wireless communication, improves response time, and enhances data privacy by avoiding the transmission of raw sensor data to the cloud. Overall, these findings underline the practical relevance of hybrid TinyML–edge architectures for IoT applications that require timely, scalable, and privacy-aware intelligence [31].

2.2.2 TinyML-Based Learning Approaches

TinyML has emerged as a practical approach for enabling machine learning inference on highly resource- constrained microcontroller units, eliminating the need for continuous cloud connectivity. Recent surveys highlight that TinyML is not defined by a single model type but rather by a collection of optimization techniques and lightweight frameworks designed to balance accuracy, memory footprint, and energy consumption under strict hardware constraints [2].

A recurring theme in TinyML research is the use of model compression strategies to make learning-based inference feasible on microcontrollers. Techniques such as quantization and pruning are widely adopted to reduce memory usage and computational cost. Quantization, which converts model parameters from floating-point to low-precision integer formats, has been shown to significantly decrease model size and inference latency with only minor accuracy degradation in many classification and pattern recognition tasks [23]. Pruning further complements this process by removing redundant parameters, resulting in compact and efficient

model representations. In practice, these techniques are widely adopted across different TinyML implementations, where models are deployed on platforms such as ESP32 and ARM Cortex-M-based microcontrollers to achieve real-time inference under strict memory and energy constraints [25], [32].

In addition to compression techniques, recent studies explore different combinations of machine learning models, deployment frameworks, and hardware platforms for TinyML applications. For instance, embedded inference frameworks such as TensorFlow Lite for Microcontrollers (TFLM) are commonly used to deploy lightweight models on devices such as ARM Cortex-M-based boards and ESP32 systems, enabling efficient execution within limited memory budgets [20]. Similarly, architecture-oriented approaches such as MicroNets propose neural network designs that are specifically optimized for microcontroller deployment by reducing parameter count and computational complexity while maintaining competitive performance [21].

Furthermore, hardware-aware strategies such as hardware–software co-design have been investigated to improve execution efficiency by distributing computational tasks between software and dedicated hardware components [33]. These approaches demonstrate that TinyML systems are not only defined by model compression, but also by the interaction between model architecture, runtime frameworks, and the underlying hardware platform [25], [32]. As a result, different implementations adopt varying design choices depending on application requirements, hardware constraints, and performance targets. This perspective also aligns TinyML within the broader cloud–edge computing continuum, where on-device intelligence complements cloud-based analytics by reducing communication overhead and enabling localized decision-making. Although TinyML has been predominantly explored in sensing and anomaly detection applications, recent localization-oriented studies emphasize the need for efficient inference strategies when handling noisy and time-varying signals [7]. In particular, RSSI-based indoor localization systems require lightweight models that can operate reliably under fluctuating signal conditions and limited computational resources. Such requirements further highlight the importance of combining model efficiency, hardware compatibility, and robust signal processing techniques within TinyML-based systems [25], [34].

This observation provides a direct transition to learning-based RSSI and BLE localization approaches discussed in the following section.

Table 2 highlights the main architectural differences between centralized cloud processing and lightweight on-device intelligence, while also summarizing the advantages and limitations of each approach for indoor localization and IoT-based systems.

Table 2: Architectural comparison of cloud, edge, and tinyML approaches

Approach	Computation Location	Data Transmission	Latency	Internet Dependency	Privacy	Typical Hardware	Model Characteristics	Main Limitation
Cloud	Remote server / centralized cloud	Raw or large-volume sensor data sent to server	High	High	Lower, since raw data leaves device	Cloud servers / data centers	Classic ML / larger models with elastic resources	Bandwidth use, transmission delay, continuous connectivity requirement
Edge	Near-data-source edge node / gateway	Reduced data transmission; partial local processing	Medium to low	Moderate	Better than cloud	Edge gateways, SBCs, local processing nodes	Moderate-size ML models; distributed analytics	Still depends on intermediate infrastructure and hardware cost
TinyML	On-device MCU	Only high-level outputs or alerts transmitted	Very low	Low or none for inference	High, since raw data remains on device	MCU-based devices such as ESP32	Lightweight, hardware-aware models (e.g., shallow CNN, MLP) with quantization/pruning	Severe SRAM/Flash, power, and compute constraints

2.2.3 RSSI and BLE-Based Indoor Localization Techniques

Indoor localization has emerged as a critical research area due to the limitations of GPS signals in enclosed environments. Comprehensive surveys consistently report that IPS must contend with signal instability, multipath propagation, and dynamic environmental conditions, all of which significantly complicate accurate position estimation. Consequently, radio frequency (RF)-based approaches have become the most widely adopted technologies for IPS, primarily due to their cost-effectiveness, scalability, and reliance on existing wireless infrastructure [35]. Among these RF-based modalities, the RSSI is extensively utilized because it can be obtained directly from standard wireless modules without additional hardware. However, the practical utility of RSSI is often hindered by its non-linear behavior and high variability, caused by fading, shadowing, and human activity within indoor spaces [5]. These physical limitations emphasize the unreliability of traditional distance-based analytical models, thereby motivating a shift toward data-driven approaches -specifically BLE- that can better capture complex signal patterns [3], [5]. BLE has gained particular prominence in this context owing to its ultra-low power consumption and widespread availability in consumer IoT devices [3]. While experimental BLE-based systems demonstrate that acceptable accuracy is achievable through strategic beacon placement, their performance remains sensitive to environmental changes, necessitating more robust estimation frameworks such as fingerprinting [36].

To specifically address the inherent instability of RSSI, fingerprint-based localization has become a dominant technique. Rather than relying on rigid signal propagation models, fingerprinting compares real-time measurements with a pre-collected database of signal patterns associated with known locations. This methodology has proven robust under NLoS conditions, particularly when integrated with machine learning-based inference and temporal aggregation to improve reliability [5], [37], [38]. Building upon this spatial robustness, recent studies have further extended RSSI analysis into the temporal domain. By incorporating time-series interpretation, researchers can extract consistent activity patterns and spatial routines even when absolute geometric accuracy is limited [38]. This shift aligns indoor localization with higher-level behavioral analytics, moving beyond mere coordinate estimation to encompass broader behavioral budgets.

Practical applications of these integrated frameworks are evident in project-level studies, such as the monitoring of livestock where BLE infrastructure and learning-based RSSI interpretation are combined into end-to-end tracking systems [37]. Such implementations demonstrate that reliable tracking is feasible through careful signal stabilization, though they often leave room for further integration with edge intelligence. This gap is also visible in open-source frameworks like 'cat-localizer,' which provides a functional infrastructure for pet tracking but remains dependent on centralized cloud servers for processing [6]. By identifying these architectural limitations, the proposed system aims to evolve this foundational work by integrating TinyML directly into the edge device (ESP32). Shifting the inference process from the cloud to the edge not only eliminates server dependency and enhances privacy but also enables real-time anomaly detection, effectively addressing the scalability and latency constraints inherent in conventional cloud-centric solutions.

2.2.4 Comparative Evaluation and Selection of Localization Methodology

The selection of a localization paradigm requires a careful trade-off between environmental complexity and system resources. While GPS serves as a primary solution for wide-area coverage, its inability to maintain line-of-sight in indoor environments necessitates alternative models [12]. Among these, triangulation and fingerprinting offer distinct advantages. Investigating the potential of triangulation in wireless networks, [19] state that this technique is more resilient to RSSI fluctuations than database-dependent 'fingerprinting' methods, especially when combined with the 'Least Square Method' to increase accuracy [13]. Furthermore, systems

using triangulation gain 'dynamic positioning' for moving targets through mathematical cleaning models like logarithmic or linear regression [14].

However, for the specific constraints of this pet tracking project, fingerprinting was selected over triangulation for the following scientific reasons:

- **Signal Noise and Obstacles:** As noted in [14], raw data in triangulation must be extensively 'cleaned' to manage signal noise caused by indoor obstacles. Fingerprinting, by contrast, implicitly incorporates these environmental reflections and shadows into the 'radio map' during the offline phase, making it more robust in complex household layouts [16], [18].
- **Hardware Efficiency:** Triangulation requires a strategic placement of sensors (maintaining angles close to 90 degrees) to narrow 'uncertainty ellipses' [15]. In a typical home environment, such precise geometric placement is often impractical. Fingerprinting allows for more flexible anchor positioning and scene analysis [17].
- **Integration with TinyML:** While triangulation relies on rigid geometric intersections [14], fingerprinting is naturally compatible with the machine learning and TinyML frameworks used in this study. This allows the system to filter out ambiguous data matches similar to the 'reconstruction and comparison' (LTRC) logic used in vision-based studies [39], but within a resource-constrained microcontroller environment [2], [40].

In conclusion, while triangulation emerges as a viable and energy-efficient alternative for high-resolution scenarios, the domestic indoor environment's unpredictable signal behavior makes the fingerprinting approach more reliable for consistent pet monitoring and anomaly detection. Table 3 summarizes representative studies related to TinyML, RSSI-based localization, edge intelligence, and anomaly detection, highlighting their methodologies, contributions, limitations, and relevance to the proposed project.

Table 3: Comparative summary of related literature

Article Name	Summary & Methodology	Key Contributions	Relevance to Project	Limitations
TinyML for Ultra-Low Power AI	Review of model compression techniques like quantization and pruning for MCUs.	Defined the resource efficiency of TinyML frameworks in large-scale IoT.	Provides the theoretical basis for model optimization on ESP32.	Focuses on general sensing, not localization.
RSSI Fingerprint-Based Localization	Uses ML algorithms to map RSSI patterns to spatial coordinates in indoor areas.	Proved that fingerprinting handles multipath effects better than path-loss models.	Guided the selection of fingerprinting as our core localization method.	Heavy reliance on cloud-based processing.
A TinyML Deep Learning Approach for Indoor Tracking of Assets (2023)	Uses BLE Tags and scanning stations with a Neural Network deployed on the edge via Edge Impulse.	Achieved 88%-94% accuracy in indoor positioning by mitigating signal multipath fading.	Directly supports our BLE/RSSI scanning architecture and edge-based ML choice.	Requires a central system for final coordination; not fully autonomous on-tag.
TinyML-Based Real-Time Activity Recognition (2025)	Implements a quantized TinyML model on Arduino/ESP units for human-object motion classification.	Proved that 91%+ accuracy is possible on battery-friendly microcontrollers for real-time events.	Validates the use of model quantization for low-power indoor monitoring.	Focuses on doorway events rather than continuous spatial tracking.
Pet Activity Tracker using XIAO BLE Sense & TinyML (2022/2024)	Predicts pet activities (rest, walk, run) using IMU data and TinyML on a wearable collar.	Established a behavioral baseline for pets using on-device inference to save energy.	Provides a benchmark for our anomaly detection (stationary vs. active) logic.	Relies heavily on IMU; lacks the spatial RSSI-based localization component.
Edge-Based Anomaly Detection in IoT Time-Series (2024)	Proposes a Self-Supervised Learning framework for identifying collective anomalies in sensor streams.	Developed a lightweight way to detect behavioral deviations without labeled datasets.	Theoretical foundation for our "Behavioral Budgeting" and anomaly detection logic.	Tested on industrial sensors; needs adaptation for BLE signal noise.

2.3 CRITICAL ANALYSIS

2.3.1 Strengths and Weaknesses of Methods

Cloud systems are powerful, offering high computational capacity; however, they require continuous internet connectivity and introduce significant transmission latency [1]. In contrast, edge computing and TinyML enable data processing at the source, providing low latency and reducing bandwidth pressure [2]. Nevertheless, the limited memory (SRAM/Flash) and constrained energy budgets of microcontrollers (MCUs) remain substantial obstacles [2], [8]. To overcome these hardware limitations, techniques such as quantization and pruning are employed to reduce model sizes for MCU deployment [23], [25]. While these strategies facilitate edge intelligence, aggressive optimization can occasionally lead to marginal accuracy degradation in complex classification tasks [8], [23]. Furthermore, in the context of connectivity, BLE offers a low-cost and energy efficient infrastructure [4]. While fingerprinting methods provide more robust results than deterministic models in multipath environments, they

remain sensitive to the non-linear behavior and instability of RSSI signals caused by environmental dynamics [3], [5], [10].

2.3.2 Divergent Approaches in Architecture and Data

There are distinct operational differences between cloud-based models and the TinyML frameworks used in edge systems. Traditional architectures necessitate raw data transmission to the cloud, whereas TinyML processes data locally and transmits only high-level insights, significantly enhancing data privacy [2], [25]. Localization strategies also diverge; distance-based models rely on rigid mathematical formulas to account for path loss, whereas data-driven fingerprinting utilizes machine learning to match real-time signals with pre-recorded databases [5], [11]. Finally, the architectural shift is evident in model selection: classic ML utilizes the elastic resources of the cloud, while TinyML adopts hardware-aware, lightweight architectures such as shallow CNNs or MLPs to operate within strict resource envelopes [8], [24].

2.3.3 Identified Research Gaps and Project Motivation

In light of the existing literature, several critical gaps emerge that justify the development of the proposed system:

- **Dynamic Recalibration:** The necessity of constantly updating fingerprint databases to adapt to physical changes (e.g., relocation of furniture) remains a major practical obstacle [5].
- **Temporal and Behavioral Depth:** Most localization studies focus on instantaneous coordinate estimation. There is a clear deficiency in extracting consistent movement patterns and spatial routines from time-series data [38].
- **Signal Stabilization under Constraints:** More research is needed on effectively stabilizing noisy RSSI signals within the strict memory and power constraints of TinyML [3], [10].
- **Optimization-Accuracy Trade-off:** The challenge of automatically balancing accuracy and resource utilization for quantization and pruning across diverse MCU architectures remains unresolved [8].
- **Niche Application Maturity:** While projects exist for cattle tracking [37] or museum navigation [36], the integration of on-device edge intelligence with behavioral anomaly detection for small-scale, dynamic scenarios—such as pet tracking in home environments—has not yet matured.

The proposed project aims to address these voids by integrating time-series anomaly detection directly on the ESP32, thereby providing a privacy-preserving, autonomous, and energy-efficient solution that moves beyond simple coordinate estimation.

2.4 SUMMARY AND CONNECTION TO THE PROPOSED PROJECT

The existing literature establishes that while BLE-based indoor positioning is a mature field, the reliance on cloud-centric processing introduces significant bottlenecks in terms of latency, bandwidth, and privacy. Furthermore, the inherent instability of RSSI signals remains a challenge that traditional deterministic models struggle to address.

The proposed project, "TinyML-Enabled Indoor Pet Tracking," directly addresses these limitations through the following strategic connections to the literature: Shift to Edge Intelligence: Unlike many current systems that depend on centralized servers, this project implements on-device inference on an ESP32 microcontroller using TinyML. This transition eliminates the "energy tax" and latency of cloud communication.

Hardware-Aware Optimization: To overcome the memory constraints of MCUs, the project will employ quantization and pruning methodologies identified in the literature. This ensures that robust machine learning models can run within the kilobyte-level limits of the hardware without sacrificing accuracy.

From Coordinates to Behavioral Analytics: While most studies focus on instantaneous position, this project extends the research by incorporating time-series anomaly detection. By analyzing signal variance and "behavioral budgets", the system moves beyond simple tracking to identify physiological or environmental distress in pets.

Environmental Robustness: Leveraging the strengths of fingerprinting over distance-based models, the project aims to mitigate signal instability through localized pattern recognition, addressing the research gap in signal stabilization under hardware constraints.

Section 3:

Methodology

3 METHODOLOGY

3.1 SYSTEM OVERVIEW

The proposed system, visualized in Figure 2, consists of one BLE beacon transmitter and three ESP32-based receiver nodes deployed in an indoor environment. The BLE beacon is attached to the pet and continuously broadcasts RSSI signals. Among the three ESP32 devices, one operates as the master node while the remaining two operate as slave nodes.

Each ESP32 receiver collects RSSI measurements transmitted by the BLE beacon. The slave nodes forward their collected RSSI data to the master ESP32, where the received values are combined with the master node's own RSSI measurements to construct an RSSI fingerprint structure.

The generated RSSI fingerprints are processed on the master ESP32 using a TinyML-based localization model to estimate the indoor location of the pet. In addition to room-level localization, the system performs feeding area monitoring to analyze the pet's feeding routine. Furthermore, time-series based stationary behavior analysis is applied to detect prolonged inactivity periods and identify potential abnormal behavioral patterns.

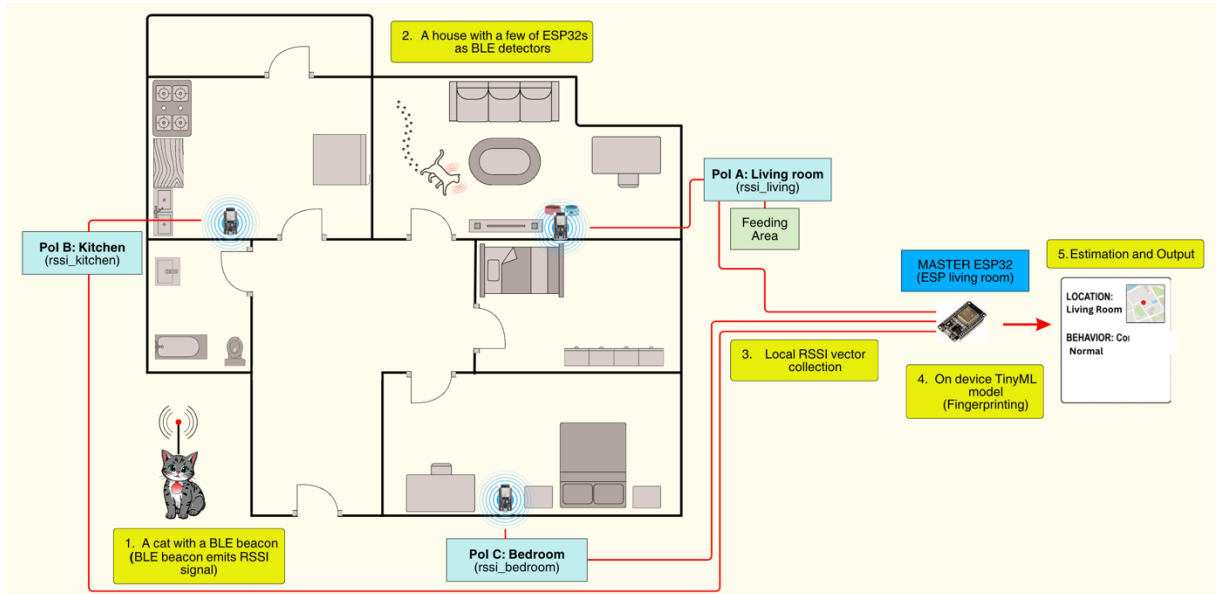


Figure 2: General system overview and applied house map

3.2 HARDWARE INFRASTRUCTURE

3.2.1 ESP32 and ESP-NOW Communication

The proposed system utilizes one ESP32-WROOM-32 board as the master node and two ESP32 development boards as slave receiver nodes. ESP32 microcontrollers were selected due to their suitability for TinyML-based edge inference and real-time IoT applications. Recent studies highlight that TinyML enables machine learning inference directly on resource-constrained Microcontroller Units (MCUs), reducing latency, communication overhead, and energy consumption in edge-oriented systems [2], [8].

In addition to their compatibility with TinyML frameworks, ESP32 devices provide integrated wireless communication capabilities such as Bluetooth and Wi-Fi, making them suitable for continuous BLE scanning and RSSI collection in indoor environments. In the proposed architecture, the two slave ESP32 nodes collect RSSI measurements transmitted by the BLE beacon and forward the collected data to the ESP32-WROOM-32 master node using the ESP-NOW communication protocol, as shown in the Figure 3.

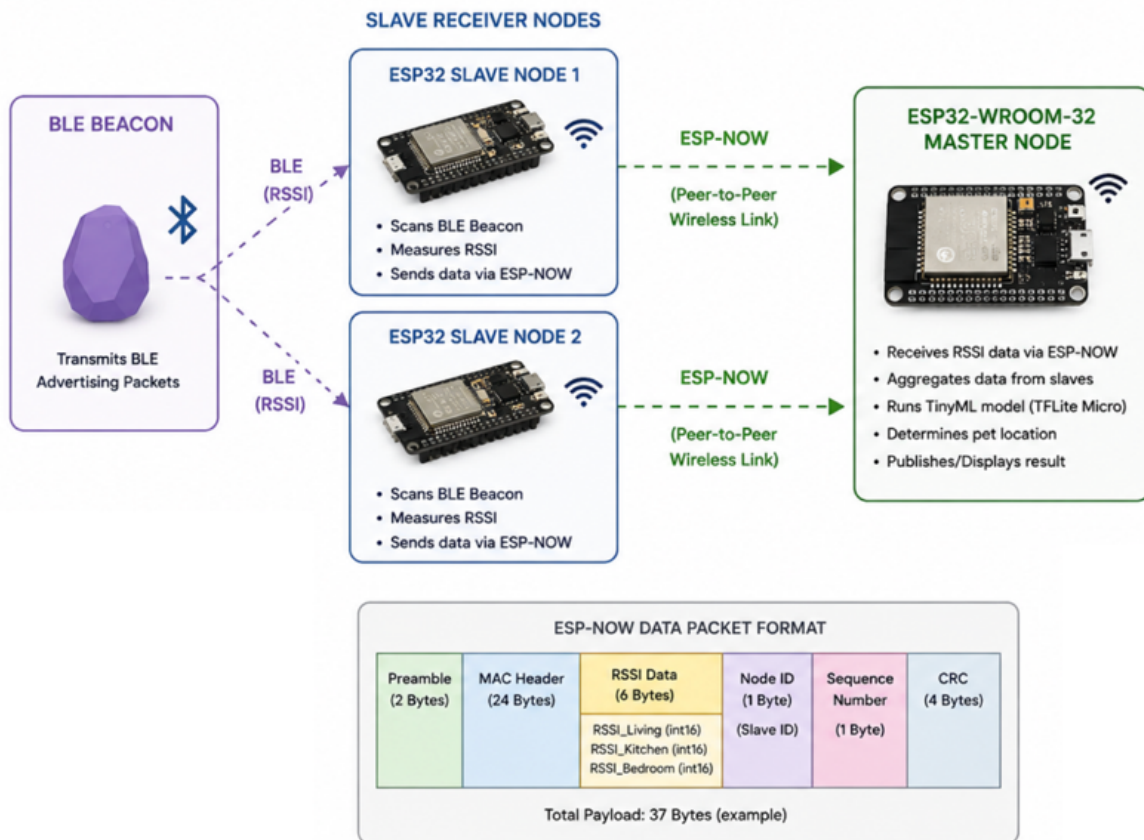


Figure 3: ESP-NOW communication architecture

ESP-NOW is a connectionless peer-to-peer wireless communication protocol developed for ESP-series microcontrollers. Unlike conventional Wi-Fi communication, ESP-NOW allows direct low-overhead data exchange between ESP32 devices without requiring router-based network establishment or continuous TCP/IP communication. This characteristic significantly reduces communication latency and power consumption while improving real-time responsiveness in distributed embedded systems. In the proposed architecture, ESP-NOW was utilized to transmit RSSI measurements collected by the slave receiver nodes to the master ESP32 node with minimal communication overhead, enabling efficient edge-oriented localization and TinyML inference.

3.2.2 BLE Technology and BLE Beacon

Bluetooth Low Energy (BLE) is a low-power wireless communication technology widely used in Internet of Things (IoT) applications due to its low energy consumption, cost efficiency, and widespread availability in consumer-grade devices. Unlike Bluetooth Classic, which is designed for continuous high-bandwidth communication, BLE focuses on periodic data transmission with significantly reduced power consumption, enabling devices to operate for extended periods using small batteries. BLE achieves this efficiency through short and intermittent transmission bursts instead of continuous streaming, making it highly suitable for lightweight IoT and indoor monitoring applications [4].

Indoor localization systems require alternative technologies because GPS-based localization performs poorly in enclosed environments due to line-of-sight limitations. These limitations have motivated the adoption of radio frequency (RF)-based localization methods, which provide higher adaptability and can operate using existing wireless communication infrastructures [5]. Among RF-based technologies, BLE has emerged as one of the most widely adopted solutions for Indoor Positioning Systems (IPS) because of its ultra-low power consumption, flexible transmission configuration, and suitability for indoor environments [3], [4].

In indoor localization systems, BLE commonly operates using beacon-based communication. BLE beacons periodically broadcast advertisement packets that can be detected by nearby scanning devices such as ESP32 receiver nodes. This advertising and scanning mechanism enables continuous RSSI collection without requiring complex communication overhead, making BLE suitable for real-time proximity sensing and indoor tracking applications [4].

Based on these advantages, a BLE beacon was selected in the proposed system to enable continuous RSSI broadcasting from the pet. This approach provides a low-cost, scalable, and energy-efficient infrastructure for RSSI fingerprint-based indoor localization and behavioral monitoring.

Additionally, the BLE beacon used in the proposed system, shown in Figure 4, was a custom-configured beacon device that provides a fixed MAC address. The use of a static device identifier simplified the receiver-side configuration and enabled consistent identification of the target beacon during RSSI collection and scanning operations. This contributed to a more stable and manageable indoor localization infrastructure, particularly during the fingerprint dataset generation process.

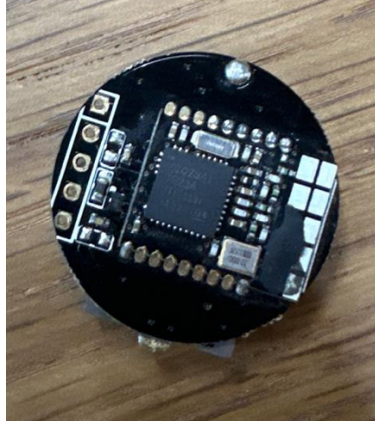


Figure 4: Custom-configured BLE beacon

3.3 DATA COLLECTION PROCESS

During the data collection stage, the master ESP32-WROOM-32 node was connected directly to a computer in order to monitor incoming RSSI measurements through the Arduino IDE Serial Monitor interface. RSSI values collected by both the slave ESP32 nodes and the master node were continuously observed and recorded during the experiments.

Initially, a stationary dataset was generated by placing the BLE beacon at predefined fixed positions inside the living room, bedroom, and kitchen environments. For each room, RSSI measurements were collected from multiple predefined session points identified as S01, S02, and S03, where each session represented a different physical location within the room in order to capture spatial RSSI variations across the indoor environment. The locations of these session points are illustrated in Table 4. During data acquisition, the BLE beacon was kept stationary at each session point for a predefined time interval while RSSI measurements from all ESP32

receiver nodes were continuously recorded. Room-level raw positioning datasets were created by generating separate raw RSSI CSV files for each room. Finally, the room-specific datasets were combined to generate the initial stationary localization dataset used during the first-stage model training process.

Table 4: The stationary data collection sessions

ROOM (FEATURE)	SESSION	NOTE
Living room	S01	sofa_near
	S02	on_the_table
	S03	near_window
	S04	feeding_area
Bedroom	S01	on_the_bed
	S02	near_table
	S03	near_the_door
Kitchen	S01	near_balcony
	S02	near_the_door
	S03	center

Each collected sample consisted of three RSSI-based features: `rss_i_living`, `rss_i_kitchen`, and `rss_i_bedroom`. These values were collected simultaneously from three ESP32 receiver nodes and combined to generate the RSSI fingerprint structure used for room-level localization.

Additionally, a separate RSSI acquisition process was conducted for feeding area monitoring. Since the feeding area was located near the living room receiver node, the BLE beacon was moved around the pet food bowl region in order to collect close-range RSSI measurements corresponding to feeding-related interactions. These measurements were recorded as an additional session (S04) during dataset generation.

To improve signal continuity during data collection, a missing signal handling mechanism was implemented on the receiver nodes. If the BLE beacon could not be detected for fewer than three consecutive scan cycles, the previously recorded valid RSSI value was preserved and reused. However, if the signal loss persisted for more than three consecutive scan cycles, a predefined RSSI value of -110 dBm was assigned to represent a no-signal condition. This approach reduced abrupt RSSI interruptions caused by temporary wireless interference and ensured more stable fingerprint generation during dataset collection.

In the second stage of dataset generation, illustrated in Table 5, a dynamic dataset was collected to better represent real-world indoor movement scenarios. Unlike the stationary dataset collection process, the BLE beacon was continuously moved within the target rooms during RSSI acquisition. Four main moving sessions were recorded and labeled as M01, M02, M03, and M04. The M01 session represented movement inside the living room, M02 represented movement inside the bedroom, and M03 represented movement inside the kitchen. The M04 session was collected around the living room feeding area and represented small-scale movements near the food bowl, which are expected during feeding-related behavior.

Table 5: The dynamic data collection session

ROOM (FEATURE)	SESSION	NOTE
Living room	M01	moving_in_room
	M04	feeding_area_small_movement
Bedroom	M02	moving_in_room
	M06	moving_in_room
Kitchen	M03	moving_in_room
	M05	moving_in_room

After the main moving sessions were collected, two additional short sessions were recorded to improve the balance of the dataset. These sessions were labeled as M05 and M06 and were collected for the kitchen and bedroom environments. Although these sessions were shorter than the main moving sessions, they were included to reduce class imbalance and increase the number of RSSI samples for rooms with fewer collected data points.

Overall, the complete dataset collection process consisted of both stationary and dynamic RSSI acquisition. The stationary sessions captured fixed-location RSSI fingerprints, while the moving sessions captured more realistic signal variations caused by indoor movement, different beacon orientations, and short-distance position changes. This combination allowed the dataset to represent both stable room-level localization conditions and movement-based scenarios that are more likely to occur during real-world pet tracking.

3.4 DATA PREPROCESSING

After the raw RSSI datasets were collected, a preprocessing pipeline was applied to prepare the data for supervised model training. First, the raw CSV files obtained from different rooms and sessions were inspected and merged into a unified localization dataset. Each sample consisted of three RSSI-based input features: *rss_i_living*, *rss_i_kitchen*, and *rss_i_bedroom*, while the target variable corresponded to the indoor location label.

Rows containing missing values in the RSSI feature columns or label column were removed to prevent incomplete samples from affecting the training process. Since temporary signal losses were already handled during RSSI acquisition on the ESP32 receiver nodes, the predefined “-110 dBm” value was preserved in the dataset as a no-signal indicator rather than being treated as a missing value.

To support feeding behavior analysis, the RSSI samples collected from the S04 feeding-area session were separated from the standard *living_room* class and assigned a dedicated *feeding_area* label. This additional class definition enabled the model to learn feeding-related proximity patterns as an independent localization category. Subsequently, all categorical location labels were converted into numerical class values using label encoding, where each localization class was mapped to a unique integer representation suitable for machine learning algorithms.

After the label encoding stage, the RSSI feature values were standardized using *StandardScaler* to ensure that all input features contributed to the model training process on a comparable numerical scale. No additional RSSI smoothing or rolling average filtering was applied during preprocessing. Instead, the natural fluctuations of indoor RSSI signals were preserved in order to reflect real-world deployment conditions and allow the model to learn from realistic signal variability.

3.5 LOCALIZATION MODEL

The proposed localization framework combines RSSI fingerprinting with a lightweight machine learning model to estimate the indoor location of the pet in real time. In this methodology, RSSI measurements collected simultaneously from multiple ESP32 receiver nodes are transformed into fingerprint vectors representing the spatial signal characteristics of the indoor environment. These fingerprints are then processed using a lightweight Multilayer Perceptron (MLP) model designed for edge-oriented TinyML deployment on resource-constrained embedded systems.

3.5.1 Fingerprinting Rationale

The proposed system utilizes an RSSI fingerprinting-based approach to estimate the pet's location. This method was selected over conventional trilateration or distance-based techniques due to the inherent complexity of indoor environments. In such settings, RSSI signals are highly susceptible to multipath propagation, reflection, shadowing, and interference, leading to significant fluctuations that render direct distance estimation unreliable [3].

Unlike geometric localization, RSSI fingerprinting does not rely on precise distance calculations. Instead, it leverages location-specific signal patterns collected from multiple receiver nodes. In this study, RSSI measurements from ESP32 nodes located in the living room, kitchen, and bedroom were synchronized and aggregated into a single fingerprint vector (comprising *rssi_living*, *rssi_kitchen*, and *rssi_bedroom* features). These vectors were then mapped to predefined location labels to characterize the unique signal environment of each room.

This approach is particularly advantageous for the proposed system for several reasons. First, it inherently accommodates indoor signal instability by focusing on spatial patterns rather than raw distance. Second, it facilitates the use of lightweight machine learning models that can learn room-dependent distributions directly from data without complex calibration. This makes the fingerprinting method highly compatible with edge-oriented TinyML frameworks, ensuring efficient operation on resource-constrained embedded devices. During the inference phase, real-time RSSI fingerprints are processed by the trained model to classify the pet's location within the indoor environment instantaneously.

3.5.2 MLP Rationale

A Multi-Layer Perceptron (MLP) classifier was selected as the primary localization model of the proposed system. MLP was preferred because indoor RSSI fingerprinting represents a nonlinear localization problem in which RSSI distributions vary significantly due to multipath propagation, signal reflection, shadowing, and wireless interference. Unlike linear classification approaches, MLP networks are capable of learning complex nonlinear relationships between RSSI fingerprint vectors and indoor location classes, making them highly suitable for RSSI-based indoor localization tasks.

A Multi-Layer Perceptron is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron performs weighted summation followed by a nonlinear activation function, enabling the network to learn nonlinear decision

boundaries from training data. In the proposed system, the input layer consisted of three RSSI-based fingerprint features: *rssi_living*, *rssi_kitchen*, and *rssi_bedroom*. These features represent the simultaneous RSSI measurements collected from the three ESP32 receiver nodes. The output layer corresponded to four localization classes: *living_room*, *kitchen*, *bedroom*, and *feeding_area*.

The implemented MLP architecture utilized a lightweight single hidden layer structure containing eight neurons (3-8-4 architecture), as illustrated in Figure 5. The Rectified Linear Unit (ReLU) activation function was used in the hidden layer due to its computational efficiency and suitability for lightweight neural network training. Model training was performed using the *MLPClassifier* implementation provided by the Scikit-learn library with a maximum iteration value of 1000 and a fixed random seed (*random_state* = 42) to ensure experimental reproducibility.

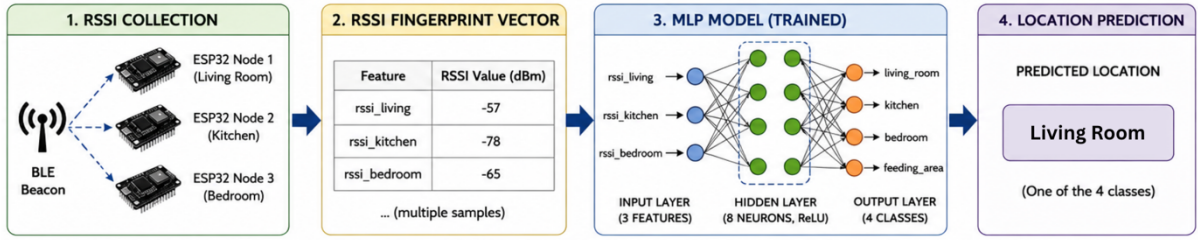


Fig. 1. Overall system pipeline: RSSI collection from multiple ESP32 nodes, fingerprint vector construction, MLP-based localization, and location prediction.

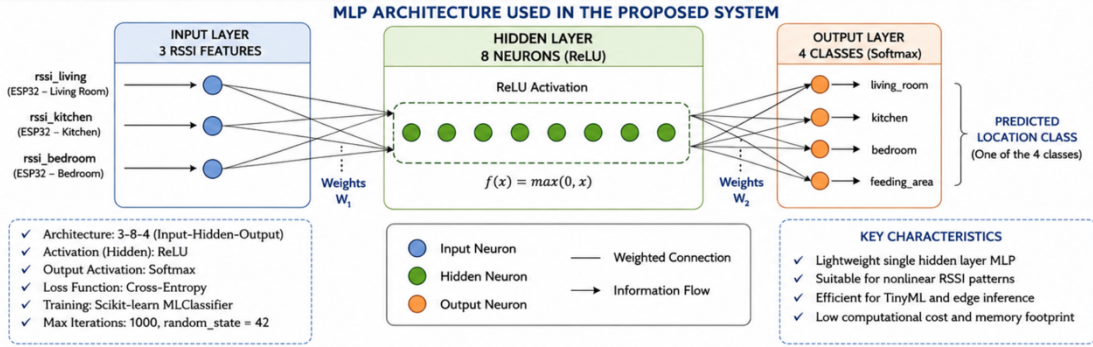


Figure 5: RSSI fingerprint-based localization pipeline and lightweight MLP architecture

During the training stage, the MLP model learned the spatial RSSI distribution patterns associated with the predefined localization labels using the fingerprint dataset generated from stationary and dynamic RSSI acquisition sessions. During inference, newly acquired RSSI fingerprint vectors were processed by the trained model to estimate the current indoor location of the pet in real time.

Several machine learning algorithms, including K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Support Vector Machine (SVM), Logistic Regression, and MLP, were

experimentally evaluated during the model selection stage. Among these approaches, MLP was selected because it provided a balance between localization performance, nonlinear learning capability, and lightweight deployment suitability for embedded TinyML systems.

In addition to its localization capability, MLP was considered particularly suitable for TinyML deployment because edge-oriented IoT applications such as localization and sensing require machine learning models with low latency, reduced energy consumption, and minimal communication overhead. Since resource-constrained Microcontroller Units (MCUs) impose strict limitations on memory usage, computational complexity, and power consumption, lightweight neural network architectures are commonly preferred in TinyML systems [2], [8]. In particular, shallow neural networks such as Multilayer Perceptrons (MLPs) are widely adopted for RSSI-based localization tasks due to their favorable trade-off between representational capability and computational cost [8], [24]. These characteristics make the proposed lightweight MLP architecture highly compatible with the ESP32-based edge inference system developed in this study.

3.6 FEEDING AREA DETECTION STRATEGY

The proposed system was designed not only for room-level localization but also for monitoring feeding-related behavior by detecting whether the pet was located near the food bowl area. During the system development process, two different feeding area detection strategies were investigated and experimentally evaluated. The first strategy utilized a hybrid rule-based approach in which feeding behavior was inferred using manually selected RSSI proximity thresholds after room-level localization. The second strategy integrated feeding area detection directly into the localization model by treating the feeding area as an independent RSSI fingerprint class. Both approaches were analyzed in terms of localization stability, implementation complexity, scalability, and suitability for real-world indoor deployment conditions.

3.6.1 Rule-Based Feeding Area Detection

During the initial system design stage, a hybrid localization strategy combining room-level localization and rule-based feeding area detection was investigated. This decision was made because the feeding area is not a separate physical room; instead, it represents a specific behavioral zone located within the living room. Therefore, the machine learning model was trained to perform room-level localization, while feeding-related behavior was detected using an additional RSSI-based proximity rule.

The final feeding area detection pipeline was implemented as a hybrid decision mechanism. First, a lightweight Multilayer Perceptron (MLP) model was trained for three-class room-level localization using the *rss_i_living*, *rss_i_kitchen*, and *rss_i_bedroom* features. The model predicted one of three room labels: *living_room*, *kitchen*, or *bedroom*. After the room prediction stage, a rule-based correction was applied only when the predicted room was *living_room*. If the model predicted *living_room* and the corresponding *rss_i_living* value was greater than feeding threshold, the final state was updated as *feeding_area*. Otherwise, the original room prediction was preserved.

To determine a suitable threshold value, multiple candidate RSSI thresholds such as -60 , -55 , -50 , -45 , and -40 dBm were experimentally analyzed using both standard living room samples and feeding-area RSSI measurements. These thresholds were evaluated according to feeding detection coverage and false alarm behavior. Feeding Area Threshold Analysis showed in the Table 6.

Table 6: Feeding Area Threshold Analysis

Threshold Value	Feeding Area Capture	False Alarm on Living Room	Interpretation
-60 dBm	High	High	Too sensitive
-55 dBm	Moderate/High	Moderate	Still causes false positives
-50 dBm	Balanced	Acceptable	Selected threshold
-45 dBm	Lower	Low	Too strict
-40 dBm	Very Low	Very Low	Misses feeding samples

Although the rule-based strategy provided a relatively simple implementation approach, several practical limitations were observed. Indoor RSSI signals are highly sensitive to environmental interference, signal reflections, beacon orientation, and short-term fluctuations, which made manually selected thresholds difficult to generalize under varying conditions. In addition, the threshold-based structure increased system complexity by introducing additional handcrafted decision logic and environment-dependent calibration requirements. These characteristics reduced the long-term scalability and adaptability of the system.

3.6.2 Machine Learning-Based Feeding Area Detection

As an alternative and based approach, the feeding area was modeled directly as an independent localization class within the RSSI fingerprint dataset. During the initial data collection stage,

RSSI samples collected from the S04 session were labeled as *living_room* and marked with feeding-area-related notes corresponding to the pet food bowl region. During dataset preprocessing, these S04 samples were separated from the standard *living_room* class and relabeled as *feeding_area*. This restructuring process enabled the localization system to learn feeding-related RSSI proximity patterns directly from the fingerprint dataset without relying on manually selected RSSI thresholds.

In this strategy, feeding area detection becomes part of the main localization task rather than an additional post-processing decision layer. The localization model learns the RSSI distribution characteristics associated with the feeding area in the same manner as other indoor room classes. Consequently, the system can directly predict *feeding_area* as one of the possible localization outputs during inference.

Experimental evaluations demonstrated that the machine learning-based feeding area classification strategy produced more stable and reliable results compared to the rule-based threshold approach. Furthermore, integrating feeding area detection directly into the localization pipeline simplified the overall system architecture by eliminating manually tuned threshold logic and additional rule-based calibration stages.

From a scalability perspective, the machine learning-based strategy also provides a more flexible foundation for future extensions. Additional behavioral zones or activity-related regions can be incorporated into the localization framework through dataset expansion and additional labels without redesigning environment-specific decision rules. Based on these advantages, the proposed system adopts the machine learning-based feeding area classification strategy as the primary feeding behavior detection method.

3.7 PROXIMITY ANALYTICS AND BEHAVIORAL BUDGETING

Temporal behavior analysis in the proposed system is implemented as a post-localization layer operating on top of the RSSI fingerprinting-based TinyML pipeline. While the embedded multilayer perceptron (MLP) model performs instantaneous location estimation using the RSSI feature vector composed of *rss_i_living*, *rss_i_kitchen*, and *rss_i_bedroom*, higher-level behavioral interpretation is achieved through temporal aggregation and rule-based analysis.

In the embedded implementation, the master ESP32 performs inference at fixed time intervals using the most recent RSSI values collected from distributed receiver nodes via BLE scanning and ESP-NOW communication. Each inference window produces a raw prediction (e.g.,

living_room, *kitchen*, *bedroom*, or *feeding_area*). However, instead of directly using these predictions, a temporal smoothing mechanism is applied.

To mitigate the effect of short-term RSSI fluctuations and noisy predictions, a majority voting scheme is employed over a sliding window of recent predictions. The most frequently occurring class within this buffer is selected as the *stable prediction*. This approach significantly improves robustness against transient misclassifications caused by signal instability, which is a well-known issue in RSSI-based indoor localization systems .

In addition to temporal smoothing, signal validity control is incorporated before model inference. RSSI values equal to -110 dBm represent missing or unreliable signals rather than actual distance measurements. Unlike earlier approaches that only checked for complete signal loss, the proposed system introduces a stricter validation mechanism. If the RSSI vector is considered unreliable—such as cases where all signals are missing or only a single weak signal is present—the system does not perform inference and instead labels the state as *no_signal*. This prevents the model from producing misleading classifications under incomplete or noisy input conditions.

Furthermore, to handle intermittent packet loss and BLE scanning delays, the system incorporates a timeout-based RSSI persistence mechanism. Recently observed RSSI values are preserved for a predefined duration instead of being immediately replaced by -110 dBm. This prevents abrupt signal drops from causing unnecessary state transitions and improves overall temporal consistency.

Building upon stable localization outputs, the system implements a behavioral interpretation layer focused on meaningful interaction detection. In particular, feeding behavior is modeled as a temporally validated event rather than an instantaneous classification. When the stable prediction indicates *feeding_area*, a candidate interaction is initiated. However, this state is only confirmed as a valid feeding event if it persists for a predefined duration (e.g., 10–15 seconds). This confirmation mechanism ensures that transient proximity such as passing near the feeding area is not misinterpreted as actual feeding behavior.

Once confirmed, the system continuously tracks feeding duration and maintains temporal metrics such as cumulative feeding time and time since the last feeding interaction. These features form the basis of a lightweight behavioral budgeting model, which enables the detection of long-term anomalies. For example, if no feeding interaction is observed within a predefined threshold (e.g., 12 hours), the system flags a contextual anomaly.

The overall temporal behavior analysis pipeline can therefore be summarized as follows:

- (i) validate RSSI signal reliability,
- (ii) perform TinyML-based localization,
- (iii) stabilize predictions using majority voting,
- (iv) detect meaningful interactions using temporal thresholds, and
- (v) derive behavioral metrics for anomaly detection.

This layered approach allows the system to distinguish between transient signal noise and meaningful behavioral patterns, providing a robust and resource-efficient solution for long-term indoor pet monitoring on edge devices.

3.8 TINYML DEPLOYMENT

TinyML enables machine learning inference directly on resource-constrained embedded devices such as microcontrollers without requiring cloud-based computation. In the proposed system, *TensorFlow Lite Micro* was utilized to deploy the trained localization model on the ESP32 master node, enabling lightweight real-time RSSI fingerprint inference directly on the edge device.

The localization model was initially trained using the *Scikit-learn MLPClassifier* implementation. However, since Scikit-learn models cannot be directly executed on *TensorFlow Lite Micro*, the trained MLP architecture was recreated in *TensorFlow* in order to enable embedded deployment compatibility.

After recreating the neural network architecture in TensorFlow, the model was converted into TensorFlow Lite (.tflite) format using the TensorFlow Lite conversion pipeline. The generated “.tflite” model was subsequently transformed into a C-style byte array and exported as a header file (model_data.h) for integration into the ESP32 firmware. This conversion process enabled the trained localization model to be embedded directly into the memory of the microcontroller for on-device inference.

After the conversion stage was completed, the *TensorFlow Lite Micro* model was integrated into the master ESP32 firmware. During runtime, the master ESP32 receives RSSI measurements from the slave ESP32 receiver nodes, combines them with its own RSSI measurement, and generates the corresponding RSSI fingerprint vector. The input vector is then standardized using the same preprocessing parameters applied during model training before being passed to the *TensorFlow Lite Micro* interpreter for real-time indoor localization inference. The overall deployment pipeline summarized in Figure 6.

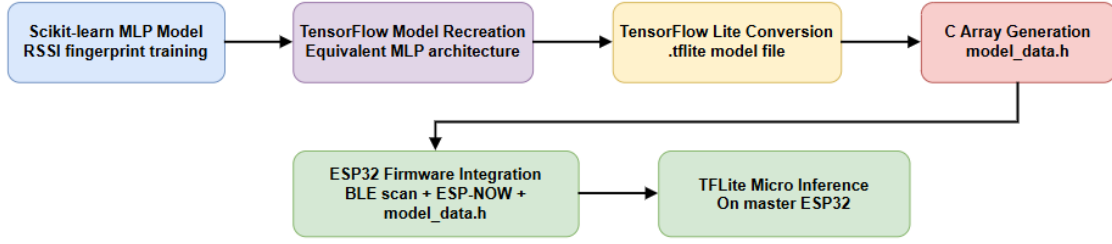


Figure 6: TinyML model deployment pipeline

This TinyML deployment approach transforms the project from an offline machine learning experiment into an edge-based intelligent monitoring system. By running inference directly on the master ESP32, the system reduces communication overhead, improves response time, and supports local decision-making for indoor pet tracking.

3.9 EVALUATION STRATEGY AND PERFORMANCE METRICS

The evaluation framework of the proposed RSSI fingerprinting-based indoor localization system was designed to analyze model behavior under multiple indoor RSSI conditions while considering both localization performance and TinyML deployment suitability. Since indoor RSSI measurements are strongly affected by multipath propagation, signal reflection, environmental dynamics, beacon orientation, and temporal wireless fluctuations, the evaluation process incorporated multiple validation strategies in order to obtain a more reliable assessment of model robustness and generalization capability.

During the model comparison stage, five machine learning approaches were evaluated: Multilayer Perceptron (MLP), K-Nearest Neighbors (KNN), Random Forest, Decision Tree, and Support Vector Machine (SVM). All models were trained using the same RSSI fingerprint structure composed of *rssi_living*, *rssi_kitchen*, and *rssi_bedroom* features, while the output variable represented the corresponding indoor localization class. This comparison enabled the proposed lightweight MLP architecture to be analyzed against classical distance-based, tree-based, and margin-based machine learning methods under identical dataset conditions.

Three complementary validation strategies were considered during the evaluation process. First, a random train-test split strategy was applied as the primary evaluation and deployment configuration in order to measure model performance on unseen RSSI samples. Second, 5-fold stratified cross-validation was utilized to obtain a more stable estimate of model behavior across

different dataset partitions while preserving class distributions. Finally, session-based evaluation was additionally analyzed to investigate the potential impact of session-dependent RSSI variations on localization performance. Since RSSI values may vary across different collection times, physical positions, beacon orientations, and environmental conditions, session-based analysis provided additional insight into the generalization capability of the proposed localization framework under realistic indoor deployment conditions.

In addition to classification performance, the evaluation process also considered TinyML deployment constraints associated with ESP32-based embedded inference. Since the final localization framework is intended to operate directly on resource-constrained microcontroller hardware, model selection was performed by considering not only predictive capability but also computational complexity, memory efficiency, and compatibility with TensorFlow Lite Micro deployment. Lightweight architectures with lower inference overhead were therefore prioritized during the model selection process.

To evaluate localization behavior comprehensively, multiple performance metrics were utilized throughout the experiments. Classification accuracy was used to measure the overall percentage of correctly predicted indoor localization samples. However, because the dataset includes multiple indoor classes with different sample distributions, including the relatively smaller *feeding_area* class, balanced accuracy was additionally considered to reduce bias originating from class imbalance. Balanced accuracy evaluates the average recall obtained across all classes and therefore provides a more reliable assessment of class-level detection performance in imbalanced localization scenarios.

Precision, recall, and F1-score metrics were further analyzed using classification reports generated for each evaluated model. Precision was used to evaluate the reliability of predicted localization labels, while recall measured the ability of the models to correctly identify samples belonging to each indoor location class. The F1-score was utilized to provide a harmonic balance between precision and recall, enabling a more stable class-level performance analysis. Finally, confusion matrix analysis was performed to examine inter-class prediction behavior and identify potential RSSI overlap between indoor environments. This analysis enabled the observation of localization confusion patterns between rooms with similar RSSI fingerprint distributions while also evaluating the distinguishability of close-range *feeding_area* interactions within the proposed behavioral monitoring framework.

Section 4:

Results

4 RESULTS

In this section, the performance of the proposed ESP32-based indoor pet tracking and anomaly detection system is evaluated in terms of different dataset structures, machine learning models, data splitting strategies, and feeding area detection performance. In order to analyze the robustness of the system, three different dataset scenarios were constructed. Dataset A consisted only of stationary RSSI samples, while Dataset B extended this structure by including feeding_area labeled behavioral data. Dataset C combined dynamic RSSI samples with feeding area data in order to create a more realistic real-world indoor monitoring scenario.

The initial experiments were conducted on Dataset A, which consisted only of stationary RSSI data. In this stage, KNN, Random Forest (RF), Decision Tree (DT), SVM, Logistic Regression, and MLP models were compared. Under the Random Sampling strategy, Random Forest and Decision Tree achieved the highest performance with approximately 96.03% accuracy. KNN achieved approximately 94.84% accuracy, while SVM reached approximately 94.44% accuracy, as shown in Figure 7. Logistic Regression produced comparatively lower performance, indicating that RSSI fingerprint distributions could not be sufficiently represented using linear decision boundaries. Although the MLP model achieved approximately 94.84% accuracy, it was selected as the primary localization model due to its low computational cost, TinyML compatibility, and strong generalization capability under different data splitting strategies.

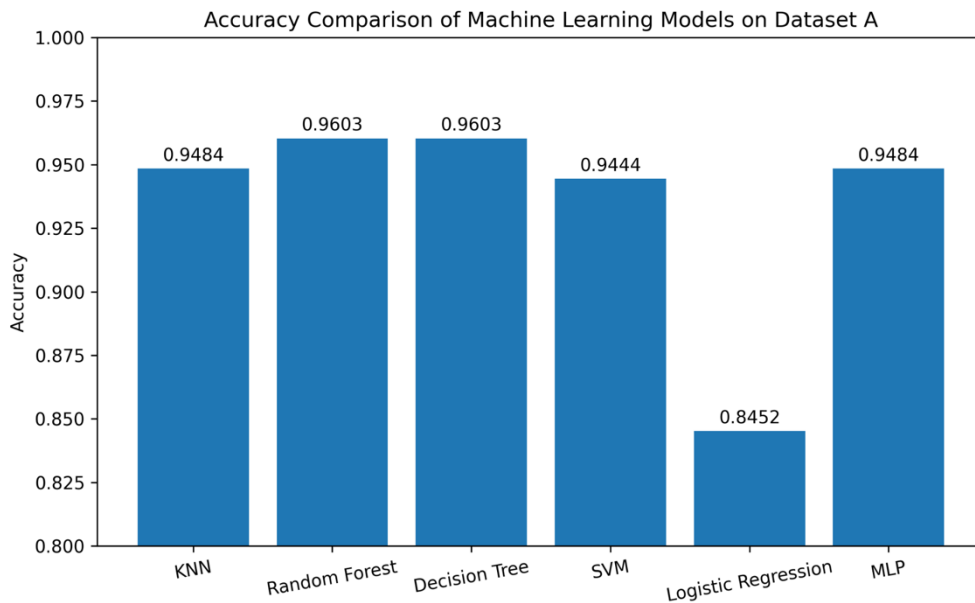


Figure 7: Accuracy comparison of evaluated machine learning models

The effect of data splitting strategies on model performance was also analyzed. Under the Random Sampling strategy, the models generally produced accuracy values between approximately 84% and 96%. This method enabled balanced dataset partitioning and allowed rapid comparison of the initial model performances. However, since Dataset A only contained stationary RSSI samples, sequential RSSI observations were highly similar to each other, which introduced a potential overfitting risk. In order to obtain a more reliable performance evaluation, 5-Fold Cross Validation was applied. Under this evaluation strategy, the MLP model achieved approximately 91.73% mean accuracy with a low standard deviation of approximately 0.007, demonstrating stable performance. When Random Sampling was evaluated under more complex dataset conditions, the overfitting risk was observed to decrease. In the session-based split approach, RSSI samples collected from sessions S1 and S2 were used as training data, while the samples collected from session S3 were used as test data. In this way, the generalization capability of the model on previously unseen RSSI distributions was evaluated. This approach provided a more realistic test scenario for analyzing the robustness of the model against RSSI variations caused by different data collection conditions. Under this evaluation strategy, the KNN model experienced a noticeable decrease in performance, while the MLP model preserved strong generalization capability with approximately 96.74% accuracy. These results indicate that the MLP model was more robust against RSSI variations occurring across different collection sessions.

The analyses conducted on the final dynamic dataset demonstrated that RSSI fingerprint structures could still be successfully distinguished under moving and unstable signal conditions. As shown in Table 7, the `living_room` class achieved approximately 95% precision and 94% recall, making it one of the most stable classes under dynamic conditions. The kitchen class produced approximately 79% precision and nearly 100% recall, while the bedroom class achieved approximately 86% F1-score. Nevertheless, increased confusion was observed between the kitchen and bedroom classes under dynamic RSSI conditions. In particular, as shown in Figure 8, several bedroom samples were classified as kitchen samples in the MLP confusion matrix. The primary reason for this behavior was the increase in RSSI fluctuations, beacon orientation changes, signal reflections, and multipath propagation effects during movement scenarios. Despite these challenges, the models preserved the fundamental RSSI fingerprint structures of different rooms and maintained an overall moderate-to-high classification performance, as illustrated in the heatmap analysis in Figure 9. In addition, representing no-signal conditions with -110 dBm contributed positively to the model's ability to manage uncertainty during room transitions.

Table 7: Classification performance metrics of the selected MLP model under dataset C

Label	Presicion	Recall	F1-Score	Support
0 (living_room)	0.95	0.94	0.95	111
1 (kitchen)	0.79	1.00	0.88	99
2 (bedroom)	0.96	0.77	0.86	111
3 (feading_area)	1.00	0.86	0.93	29

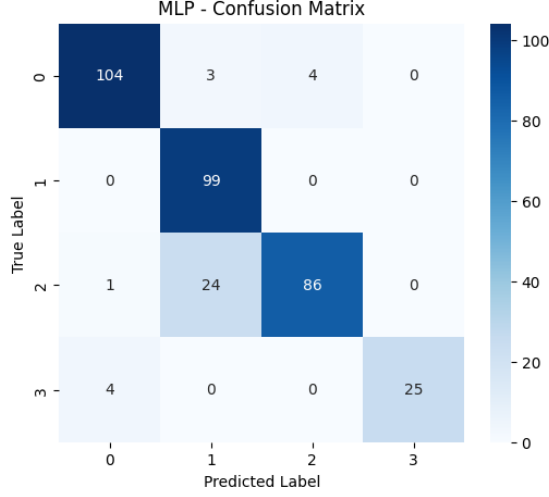


Figure 8: Confusion matrix of the selected MLP model under dataset C.

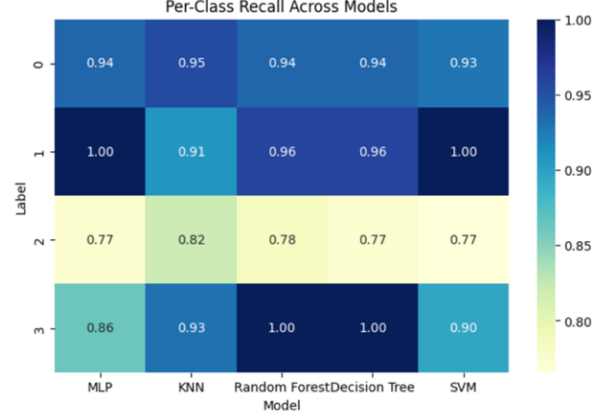


Figure 9: Per-class recall comparison of MLP, KNN, random forest, decision tree, and SVM models under datasets

Feeding area detection became one of the most important components of the proposed anomaly-aware monitoring system. Initially, an RSSI threshold-based rule-based approach was evaluated. Later, feeding_area was directly modeled as an independent fingerprint class and an MLP-based fingerprinting approach was applied. Although both approaches achieved similar performance levels, the fingerprinting-based MLP approach achieved approximately 4% higher accuracy, as illustrated in Figure 10. Therefore, the MLP-based fingerprinting method was selected for the final system due to its ability to reduce manual threshold tuning requirements and integrate directly into the localization pipeline. In the experiments conducted on Dataset B and Dataset C, the MLP model achieved recall values above 90% for the feeding_area class during training, while maintaining approximately 77% recall under test conditions, as shown in Figure 11. Recall was considered particularly important for feeding area detection because the primary objective of the system was to minimize missed feeding events. The obtained results demonstrated that the proposed model could detect feeding area interactions with acceptable recall performance and successfully distinguish close-range behaviors around the feeding bowl

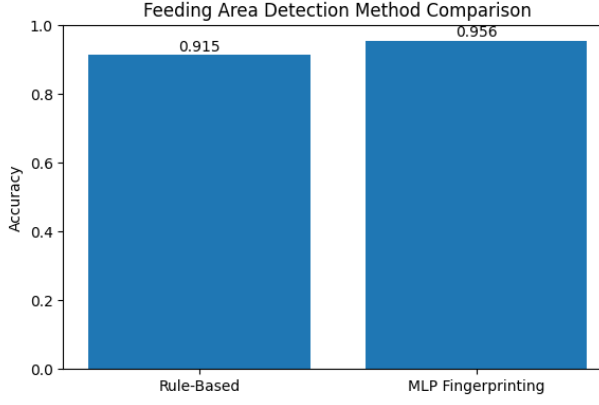


Figure 10: Feeding area detection method comparison under dataset B with random sampling

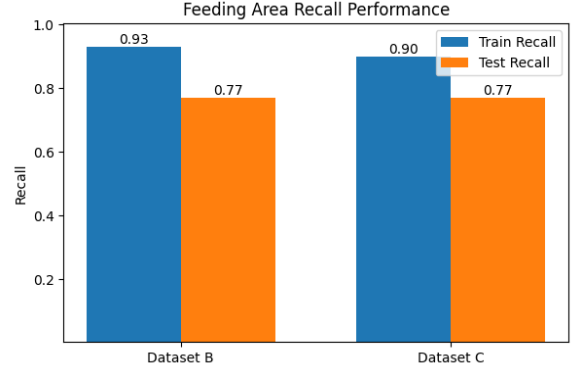


Figure 11: Feeding area recall performance comparison under dataset B and dataset C

Overall, the MLP model combined with the TinyML approach was determined to be the most balanced solution in terms of accuracy, generalization capability, and embedded system compatibility. While Dataset A achieved approximately 67.6% accuracy using only stationary RSSI data, the inclusion of feeding area samples in Dataset B increased the localization accuracy to approximately 77.6%. As illustrated in Figure X, Dataset C, which represented the most realistic scenario with dynamic RSSI conditions, achieved approximately 80.1% accuracy. These results demonstrate that the proposed MLP-based RSSI fingerprinting system could maintain stable generalization capability despite increasing dataset complexity and dynamic RSSI variations. In particular, the results obtained from Dataset C confirm that the proposed system provides a practical and reliable infrastructure for real-time indoor pet tracking, room-level localization, and behavior-aware feeding area detection.

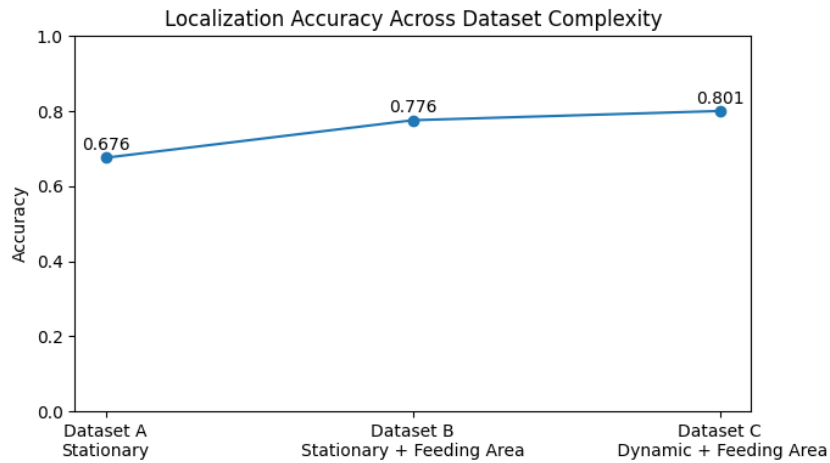


Figure 12: Localization accuracy results across all datasets obtained from experimental te

In addition to room-level localization, a lightweight temporal behavior analysis layer was implemented on the ESP32 edge device. During the 4-hour experimental observation period, the

proposed temporal behavior analysis layer successfully identified two separate feeding interactions and continuously monitored the cumulative feeding duration over time. As illustrated in the feeding interaction timeline, as illustrated in Figure 13, the system was able to distinguish feeding events from normal indoor movement patterns and track the temporal continuity of feeding behavior in real time. Furthermore, the cumulative feeding duration graph, as shown in the Figure 14, demonstrates that the total feeding interaction time reached approximately 8 minutes during the observation window. These results show that the proposed ESP32-based framework can transform instantaneous feeding_area predictions into higher-level behavioral indicators suitable for long-term behavioral monitoring and anomaly-aware pet tracking applications.

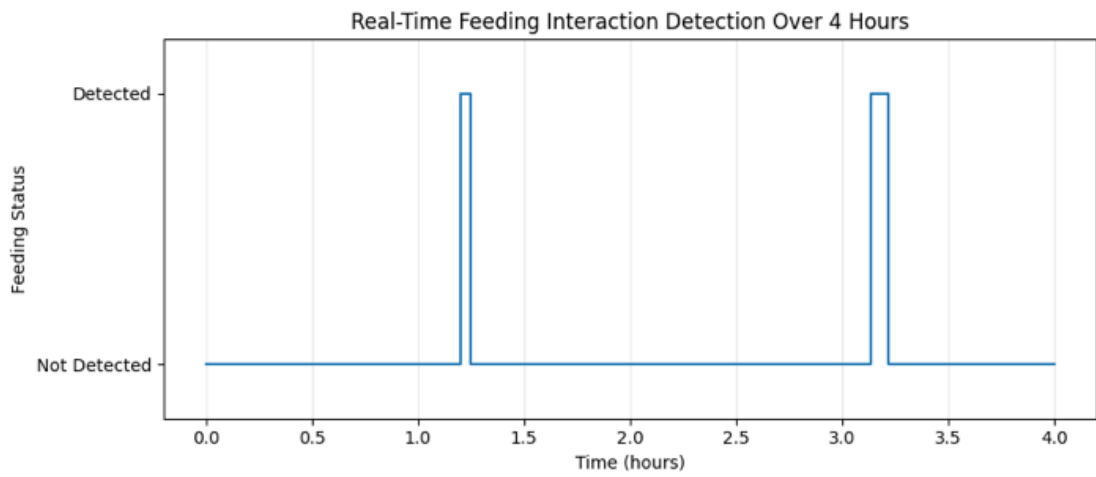


Figure 13: Real-time feeding interaction timeline over a 4-hour observation period

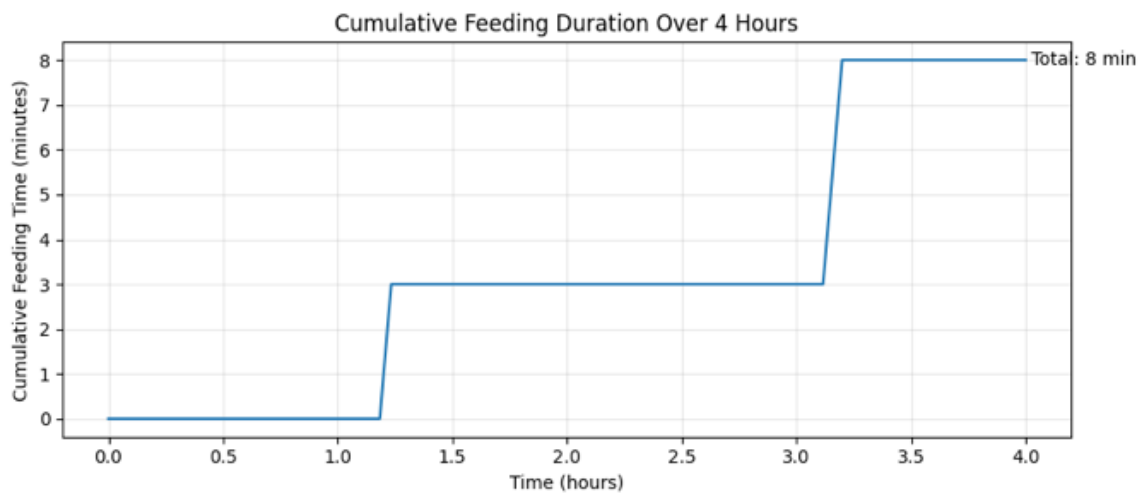


Figure 14: Cumulative feeding duration during a 4-hour observation period

Section 5:

Discussion and Conclusion

5 DISCUSSION AND CONCLUSION

5.1 DISCUSSION OF LOCALIZATION PERFORMANCE

The experimental results demonstrate that the proposed RSSI fingerprinting-based localization system can distinguish indoor locations by learning signal patterns collected from multiple ESP32 receiver nodes. Since each input sample consists of three RSSI features, namely *rssi_living*, *rssi_kitchen*, and *rssi_bedroom*, the localization model does not rely on direct distance estimation. Instead, it learns room-dependent RSSI fingerprint distributions generated by the BLE beacon within the indoor environment. This supports the methodological decision to use fingerprinting rather than GPS, triangulation, or direct distance-based localization, since indoor RSSI values are highly affected by multipath propagation, signal reflection, obstacles, beacon orientation, and environmental variability.

According to the model comparison results, the evaluated machine learning algorithms achieved different levels of localization performance under the selected validation strategies. In the stationary Dataset A scenario, Random Forest and Decision Tree achieved the highest classification accuracy with approximately 96.03% under random sampling. KNN achieved approximately 94.84%, SVM achieved approximately 94.44%, and MLP also reached approximately 94.84% accuracy. However, the final model selection was not based only on the highest offline accuracy. Since the proposed system targets real-time TinyML deployment on an ESP32 microcontroller, model complexity, memory efficiency, inference feasibility, and compatibility with TensorFlow Lite Micro were also considered.

Although Random Forest and Decision Tree achieved strong classification performance in the stationary random sampling scenario, they are less suitable for direct microcontroller deployment due to higher memory or inference-time requirements. In contrast, the lightweight MLP model provides a more balanced solution by combining nonlinear learning capability with a compact architecture. The selected MLP architecture, consisting of three input features, one hidden layer with eight neurons, and four output classes, is simple enough for edge-oriented deployment while still being capable of learning nonlinear RSSI-location relationships.

The confusion matrix results further indicate that most localization classes were distinguishable based on their RSSI fingerprint characteristics. However, increased confusion was observed between the kitchen and bedroom classes under dynamic RSSI conditions. This behavior is expected in BLE RSSI-based indoor localization because RSSI signals do not always change linearly with physical distance. Similar RSSI distributions may occur when rooms are close to each other, when the beacon orientation changes, or when furniture and walls cause signal

reflections. Therefore, these misclassifications do not necessarily indicate a failure of the model; rather, they reflect the inherent difficulty of RSSI-based indoor localization in real-world environments.

In the final dynamic dataset scenario, the model preserved the fundamental RSSI fingerprint structures of different rooms despite movement-based signal variations. The *living_room* class achieved approximately 95% precision and 94% recall, making it one of the most stable classes under dynamic conditions. The *kitchen* class achieved approximately 79% precision and nearly 100% recall, while the *bedroom* class achieved approximately 86% F1-score. The *feeding_area* class also remained distinguishable, achieving high precision and acceptable recall despite having fewer samples than the main room classes. These findings show that the proposed RSSI fingerprinting approach can maintain practical localization performance under more realistic indoor movement and signal fluctuation conditions.

5.2 DISCUSSION OF VALIDATION STRATEGIES

The evaluation process included multiple validation strategies in order to better understand model behavior under different data separation conditions. The random train-test split provided a baseline performance estimate by evaluating the models on unseen samples drawn from the same overall dataset distribution. This strategy was useful for initial model comparison; however, it may produce optimistic results in RSSI fingerprinting tasks because samples collected under similar session conditions can appear in both training and testing subsets.

To obtain a more stable performance estimate, 5-fold stratified cross-validation was also applied. This strategy preserved the class distribution across folds and reduced the dependency on a single random split. The cross-validation results showed that MLP achieved an average accuracy of approximately 91.73% with a standard deviation of about 0.007, indicating stable performance across different dataset partitions.

Session-based evaluation provided a more realistic assessment of generalization capability. Unlike random splitting, session-based evaluation separates training and testing data according to data collection sessions. This is particularly important for RSSI fingerprinting because signal values may vary across different collection times, beacon orientations, physical positions, and environmental conditions. In this evaluation, S1 and S2 sessions were used for training, while S3 was used for testing. The session-based results showed that MLP maintained strong performance with approximately 96.74% accuracy, while KNN showed a more noticeable performance decrease. This outcome suggests that the MLP model was more robust against session-dependent RSSI variations.

Overall, the validation results confirm that RSSI fingerprinting can support indoor pet localization, but they also highlight the importance of collecting diverse training data. Including both stationary and dynamic sessions improved the dataset’s ability to represent realistic movement conditions. However, future deployments may still require additional data collection if the physical environment changes significantly, such as receiver relocation, furniture rearrangement, or beacon replacement.

5.3 *DISCUSSION OF TINYML DEPLOYMENT*

The TinyML deployment stage transforms the proposed system from an offline machine learning experiment into an embedded edge-based localization framework. In the proposed architecture, the slave ESP32 nodes collect RSSI values and transmit them to the master ESP32 using ESP-NOW communication. The master node combines these values with its own BLE RSSI measurement, applies the same preprocessing transformation used during training, and performs inference using the deployed TensorFlow Lite Micro model.

This deployment approach is consistent with the main motivation of TinyML-based IoT systems: reducing cloud dependency, lowering communication overhead, improving response time, and supporting local decision-making. Instead of sending raw RSSI data to a remote server, the proposed system performs localization directly on the ESP32 master node. This design improves privacy and makes the system more suitable for home-based pet monitoring scenarios where continuous internet connectivity may not always be available.

The conversion process from Scikit-learn MLP to TensorFlow, then to TensorFlow Lite format, and finally to a C-style header file enabled the trained model to be embedded into the ESP32 firmware. During real-time operation, the system produced localization outputs through the Serial Monitor based on incoming RSSI fingerprints. This result demonstrates that the proposed MLP-based localization model can be integrated into an ESP32-based TinyML inference pipeline.

However, TinyML deployment also introduced practical limitations. Since ESP32 microcontrollers have restricted memory and computational capacity, the localization model had to remain lightweight. This constraint supports the decision to use a shallow MLP rather than a more complex model. In addition, the system must apply the same StandardScaler parameters during real-time inference, which increases implementation complexity but is necessary to ensure consistency between training and deployment conditions.

Another deployment-related challenge is signal validity. During BLE scanning, temporary packet loss or receiver-side detection failure may produce -110 dBm values, representing a no-

signal condition. If all receiver nodes report -110 dBm, the model may still force the input into one of the trained classes unless a validity control mechanism is applied. For this reason, the proposed system includes a no-signal handling strategy before localization inference. This mechanism improves reliability by preventing invalid RSSI fingerprints from being interpreted as meaningful indoor positions.

To address RSSI instability more effectively, a multi-layer stabilization strategy was introduced. First, unreliable RSSI vectors were filtered out using a signal validation mechanism that prevents inference when the input data is incomplete or dominated by no-signal values. Second, a majority voting scheme over consecutive predictions was applied to reduce the impact of transient misclassifications. These approaches improved the temporal consistency of localization results without increasing model complexity.

5.4 DISCUSSION OF FEEDING AREA AND BEHAVIORAL MONITORING

One of the main contributions of the proposed system is that it extends indoor localization beyond simple room prediction. In addition to estimating the pet’s room-level location, the system also includes feeding area detection and stationary behavior analysis. This design aligns with the broader project goal of monitoring pet behavior rather than only identifying physical location.

During development, two feeding area detection strategies were considered. The first strategy used a rule-based threshold after room-level localization. In this approach, the system first predicted the room and then classified the state as *feeding_area* if the living room RSSI exceeded a selected threshold. Although this method was simple, it depended heavily on manually selected RSSI thresholds. Since RSSI values fluctuate due to environmental conditions, beacon orientation, signal reflection, and receiver placement, a fixed threshold may not generalize well across different homes or even different time periods in the same home.

The second strategy treated *feeding_area* as an independent class within the localization dataset. This approach allowed the model to learn feeding-related proximity patterns directly from collected RSSI fingerprints. Compared with the initial rule-based threshold approach, the machine learning-based feeding area strategy provides a more scalable and adaptive solution because it avoids manually selected RSSI thresholds and environment-dependent calibration. In the experiments conducted on Dataset B and Dataset C, the MLP model achieved recall values above 90% for the *feeding_area* class during training and maintained approximately 77% recall under test conditions. Since the main objective of feeding area detection is to reduce missed feeding events, recall was considered particularly important for this behavioral zone.

Stationary behavior analysis was designed as a temporal layer built on top of localization outputs. Instead of interpreting each prediction independently, the system monitors consecutive predictions across inference windows. If the same predicted state persists for a predefined duration, the pet is considered stationary in that region. This approach is useful because abnormal pet behavior is often not represented by a single location prediction, but by sustained changes in movement patterns, such as prolonged inactivity or unusually long dwell time near a specific area.

The proposed system also uses simple temporal rules to interpret feeding-related behavior. Feeding behavior is not inferred from a single *feeding_area* prediction; instead, it is confirmed when the *feeding_area* state persists over a predefined time threshold. This design helps distinguish intentional interaction with the food bowl from incidental proximity or transient misclassification.

Furthermore, the system introduces a lightweight behavioral budgeting mechanism that tracks cumulative feeding duration and time since the last feeding event. This enables the detection of contextual anomalies, such as prolonged absence from the feeding area. Unlike complex anomaly detection models, this method uses simple temporal rules built on top of stable localization outputs, making it suitable for embedded deployment.

The proposed behavioral monitoring approach remains preliminary, but it provides an important foundation for future anomaly detection. By combining localization outputs with temporal analysis, the system can eventually construct behavioral baselines such as visit frequency, dwell time, and routine movement patterns. This direction extends the system from indoor tracking toward context-aware pet behavior monitoring.

5.5 COMPARISON WITH EXISTING LITERATURE

The findings of this study are consistent with previous research emphasizing the limitations of GPS and deterministic distance-based localization in indoor environments. Since GPS requires line-of-sight satellite communication, it is unsuitable for home-based pet monitoring. Similarly, triangulation and path-loss-based distance estimation are sensitive to obstacles, signal reflection, and indoor multipath effects. The proposed system addresses these limitations by using RSSI fingerprinting, which learns environment-specific signal patterns rather than attempting to calculate exact geometric distances.

The results also support the literature's claim that BLE is suitable for low-power indoor localization. BLE beacons provide a practical infrastructure for continuous RSSI broadcasting, while ESP32 receiver nodes offer a low-cost platform for distributed sensing. However, the

observed RSSI fluctuations and occasional no-signal values confirm that BLE-based localization remains challenging in real-world environments. Therefore, the system’s missing signal handling, signal validation, and majority voting mechanisms are necessary components rather than optional improvements.

From the TinyML perspective, the proposed system follows the literature’s emphasis on edge-based intelligence. Prior studies highlight that cloud-based IoT architectures may introduce latency, bandwidth usage, privacy concerns, and internet dependency. By deploying inference directly on the ESP32 master node, this project reduces the need for continuous cloud communication and supports real-time local decision-making. This makes the system more aligned with privacy-preserving and low-latency IoT applications.

Compared with existing BLE tracking systems and related pet or animal monitoring studies, the proposed project contributes by combining RSSI fingerprinting, TinyML deployment, and behavioral monitoring in a small-scale indoor pet tracking scenario. While many systems focus mainly on localization, this project also considers feeding behavior and stationary activity as behavior-oriented extensions. Therefore, the system addresses the literature gap related to moving from coordinate estimation toward behavioral analytics.

5.6 LIMITATIONS

Despite the promising results, the proposed system has several limitations. First, the dataset was collected in a limited indoor environment consisting of predefined rooms and receiver placements. Therefore, the trained model is environment-specific and may not generalize directly to different houses without additional data collection or recalibration. Since RSSI fingerprinting depends on the signal characteristics of the deployment area, changes in room layout, furniture placement, wall structure, or receiver position may affect localization performance.

Second, RSSI measurements are inherently unstable. Although the system includes missing signal handling and no-signal control mechanisms, short-term fluctuations may still affect model predictions. The use of -110 dBm as a no-signal indicator helps represent signal loss, but frequent no-signal cases can reduce the reliability of real-time localization. Future versions of the system should further improve input stability using rolling average filters, temporal smoothing, majority voting, or confidence-based prediction control.

Third, the *feeding_area* class depends on the quality and diversity of close-range RSSI samples collected around the food bowl. If the feeding area position changes or if the pet interacts with

the bowl from different orientations, the RSSI pattern may shift. Therefore, feeding area detection may require additional training samples under different real-world conditions.

Fourth, the stationary behavior analysis was designed as a post-localization monitoring layer, but long-term behavioral anomaly detection requires extended observation data. Since abnormal behavior depends on each pet's normal routine, a personalized baseline must be constructed over time. Without long-term real pet data, the system can detect stationary states, but its ability to identify medically or behaviorally meaningful anomalies remains limited.

Finally, although the TinyML deployment demonstrates the feasibility of edge-based inference, the ESP32 platform imposes memory and computation constraints. These constraints limit the complexity of models that can be deployed. Therefore, future work should evaluate quantization, memory usage, inference latency, and power consumption in more detail to optimize the system for continuous operation.

5.7 CONCLUSION

This study presented a TinyML-enabled indoor pet tracking and behavioral monitoring system based on BLE RSSI fingerprinting and ESP32-based edge inference. The proposed system uses one BLE beacon attached to the pet and three ESP32 receiver nodes deployed in an indoor environment. RSSI measurements collected from the receiver nodes are combined into fingerprint vectors and processed by a lightweight machine learning model to estimate the pet's indoor state.

The project adopted RSSI fingerprinting instead of GPS, triangulation, or direct distance estimation because indoor environments introduce significant signal instability, multipath propagation, reflection, and non-line-of-sight conditions. By learning location-specific RSSI patterns, the proposed model provides a more suitable solution for indoor localization in a home environment. The collected dataset included both stationary and dynamic sessions, allowing the system to represent fixed-location fingerprints as well as movement-based RSSI variations.

Several machine learning models were evaluated, including MLP, KNN, Random Forest, Decision Tree, SVM, and Logistic Regression. The results showed that Random Forest and Decision Tree achieved the highest stationary random sampling accuracy with approximately 96.03%, while MLP achieved approximately 94.84% in the same scenario. However, because the final system targets ESP32-based TinyML deployment, the selected model was chosen not only for classification performance but also for lightweight inference suitability. The MLP architecture provided a practical balance between nonlinear RSSI pattern learning and embedded deployment feasibility. Moreover, MLP achieved approximately 91.73% mean

accuracy with 0.007 standard deviation in 5-fold cross-validation and approximately 96.74% accuracy in session-based evaluation, supporting its selection as the final model.

The final dynamic dataset scenario demonstrated that RSSI fingerprint structures could still be distinguished under more realistic movement and signal fluctuation conditions. Although dynamic data increased confusion between some classes, especially kitchen and bedroom, the system maintained approximately 80.1% overall accuracy in Dataset C. This result indicates that the proposed approach remains applicable beyond purely stationary RSSI collection conditions.

The system also extended room-level localization by incorporating feeding area detection and stationary behavior analysis. Feeding area detection was ultimately modeled as an independent fingerprint class, allowing the system to learn proximity-based feeding patterns directly from RSSI data. The *feeding_area* class achieved above 90% recall in training data and approximately 77% recall under test conditions. Stationary behavior analysis was designed as a temporal monitoring layer that observes consecutive localization predictions to identify prolonged inactivity. These additions allow the system to move beyond simple localization and toward behavior-aware pet monitoring.

The TinyML deployment pipeline further demonstrated the feasibility of running localization inference directly on the ESP32 master node. By converting the trained model into TensorFlow Lite format and integrating it into the ESP32 firmware, the system supports local inference without relying on cloud-based processing. This edge-centric design reduces communication overhead, improves response time, and supports privacy-preserving operation.

In conclusion, the proposed system demonstrates that BLE RSSI fingerprinting combined with lightweight TinyML inference can provide a feasible foundation for indoor pet localization and basic behavioral monitoring. Although RSSI instability, environment-specific calibration, and long-term anomaly validation remain important challenges, the developed framework shows strong potential for low-cost, privacy-aware, and real-time indoor pet tracking. Future work should focus on expanding the dataset across different indoor environments, improving temporal filtering, evaluating power consumption and inference latency, and validating behavioral anomaly detection using long-term real pet activity data.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor for the valuable guidance, constructive feedback, and continuous support provided throughout the development of this capstone project. Their suggestions helped us improve both the technical implementation and academic structure of our work.

We would also like to thank the Department of Computer Engineering at Istinye University for providing the academic environment and resources necessary to complete this project. This study allowed us to apply our knowledge in embedded systems, machine learning, TinyML, and IoT-based system design to a real-world indoor tracking problem.

Finally, we would like to thank our project team members for their collaboration, effort, and dedication during the research, implementation, testing, and reporting stages of the project.

APPENDIX

The source code and related project files developed within the scope of this capstone project are available in the following GitHub repository:

[TinyML Indoor Pet Tracking GitHub Repository](#)

REFERENCES

- [1] F. C. Andriulo, M. Fiore, M. Mongiello, E. Traversa, and V. Zizzo, “Edge computing and cloud computing for Internet of Things: A review,” *Informatics*, vol. 11, no. 4, Sep. 2024, doi: 10.3390/informatics11040071.
- [2] N. Schizas, A. Karras, C. Karras, and S. Sioutas, “TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review,” *Future Internet* 2022, vol. 14, no. 12, Dec. 2022, doi: 10.3390/fi14120363.
- [3] L. Bai, F. Ciravegna, R. Bond, and M. Mulvenna, “A low cost indoor positioning system using bluetooth low energy,” *IEEE Access*, vol. 8, pp. 136858–136871, Jul. 2020, doi: 10.1109/access.2020.3012342.
- [4] G. Koulouras, S. Katsoulis, and F. Zantalis, “Evolution of bluetooth technology: BLE in the IoT ecosystem,” *Sensors (Basel)*, vol. 25, no. 4, p. 996, Feb. 2025, doi: 10.3390/S25040996.

- [5] B. Zholamanov *et al.*, “RSSI fingerprint-based indoor localization solutions using machine learning algorithms: A comprehensive review,” *Smart Cities 2025*, vol. 8, no. 5, Sep. 2025, doi: 10.3390/smartcities8050153.
- [6] F. Stefaniak, “filipsPL/cat-localizer: Localize your cat at home with BLE beacon, ESP32s, and Machine Learning.” Accessed: Jan. 09, 2026. [Online]. Available: <https://github.com/filipsPL/cat-localizer>
- [7] D. Avellaneda, D. Mendez, and G. Fortino, “A TinyML Deep Learning Approach for Indoor Tracking of Assets †,” *Sensors*, vol. 23, no. 3, Feb. 2023, doi: 10.3390/s23031542.
- [8] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, “A comprehensive survey on tinyML,” *IEEE Access*, vol. 11, pp. 96892–96922, Jul. 2023, doi: 10.1109/access.2023.3294111.
- [9] S. G. Leitch *et al.*, “On indoor localization using WiFi, BLE, UWB, and IMU technologies,” *Sensors 2023*, vol. 23, no. 20, Oct. 2023, doi: 10.3390/s23208598.
- [10] S. Sadowski and P. Spachos, “RSSI-based indoor localization with the Internet of Things,” *IEEE Access*, vol. 6, pp. 30149–30161, Jun. 2018, doi: 10.1109/access.2018.2843325.
- [11] N. Singh, S. Choe, and R. Punmiya, “Machine learning based indoor localization using Wi-Fi RSSI fingerprints: An overview,” *IEEE Access*, vol. 9, pp. 127150–127174, Sep. 2021, doi: 10.1109/access.2021.3111083.
- [12] A. Pangestu *et al.*, “Pet tracking system using GPS with android-based geofencing method,” in *ICITDA 2023- Proceedings of the 2023 8th International Conference on Information Technology and Digital Applications*, IEEE, 2023, pp. 1–7. doi: 10.1109/icitda60835.2023.10427488.
- [13] E. Alp, T. Dag, and T. Arsan, “Indoor positioning system by using triangulation algorithm,” *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, no. 8, pp. 2278–3075, 2016, Accessed: May 15, 2026. [Online]. Available: www.ijitee.org
- [14] S. A. Kharmeh, E. Natsheh, R. Nasrallah, and M. Masri, “Triangulation-enhanced WiFi-based autonomous localization and navigation system: A low-cost approach,” in *2024 22nd International Conference on Research and Education in Mechatronics, REM 2024*, IEEE, 2024, pp. 69–74. doi: 10.1109/rem63063.2024.10735691.

- [15] O. Tekdas and V. Isler, "Sensor placement for triangulation-based localization," *IEEE Transactions on Automation Science and Engineering*, vol. 7, no. 3, pp. 681–685, Jul. 2010, doi: 10.1109/tase.2009.2037135.
- [16] M. J. C. S. Reis, "Lightweight signal processing and edge AI for real-time anomaly detection in IoT sensor networks," *Sensors* 2025, vol. 25, no. 21, Oct. 2025, doi: 10.3390/s25216629.
- [17] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine learning for anomaly detection: A systematic review," *IEEE Access*, vol. 9, pp. 78658–78700, May 2021, doi: 10.1109/access.2021.3083060.
- [18] Y. Kalyani and R. Collier, "A systematic survey on the sole of cloud, fog, and edge computing combination in smart agriculture," *Sensors* 2021, vol. 21, no. 17, Sep. 2021, doi: 10.3390/s21175922.
- [19] V. Osa, J. Matamales, J. F. Monserrat, and J. López, "Localization in wireless networks: The potential of triangulation techniques," *Wireless Personal Communications* 2012, vol. 68, no. 4, pp. 1525–1538, Feb. 2012, doi: 10.1007/s11277-012-0537-2.
- [20] R. David *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, Mar. 2021.
- [21] C. Banbury *et al.*, "Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers," *Proceedings of machine learning and systems*, pp. 517–532, 2021, Accessed: May 15, 2026. [Online]. Available: <https://github.com/ARM-software/ML-zoo>.
- [22] S. Ghamari *et al.*, "Quantization-guided training for compact tinyml models," Mar. 2021. Accessed: May 15, 2026. [Online]. Available: <https://arxiv.org/pdf/2103.06231>
- [23] C. R. Banbury *et al.*, "Benchmarking tinyML systems: Challenges and direction," Mar. 2020. Accessed: Jan. 08, 2026. [Online]. Available: <https://arxiv.org/pdf/2003.04821>
- [24] V. M. Oliveira and A. H. J. Moreira, "Edge AI system using a thermal camera for industrial anomaly detection," in *Social-Informatics and Telecommunications Engineering, LNICST*, Springer, Cham, Jun. 2022, pp. 172–187. doi: 10.1007/978-3-031-06371-8_12
- [25] S. Somvanshi *et al.*, "From tiny machine learning to tiny deep learning: A survey," *ACM Comput. Surv.*, vol. 58, no. 7, pp. 1–33, Dec. 2025, doi: 10.1145/3776588;
- [26] F. Milano *et al.*, "BLE-based indoor localization: Analysis of some solutions for performance improvement," *Sensors* 2024, vol. 24, no. 2, Jan. 2024, doi: 10.3390/s24020376.

- [27] A. Benmachiche, K. Rais, and H. Slimi, "Real-time machine learning for embedded anomaly detection," Dec. 2025. Accessed: Jan. 08, 2026. [Online]. Available: <https://arxiv.org/pdf/2512.19383>
- [28] M. Sri Vidya and G. R. Sakthidharan, "Accurate anomaly detection using various Machine Learning methods for IoT devices in indoor environment," *2021 Fifth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, pp. 308–316, Nov. 2021, doi: 10.1109/I-SMAC52330.2021.9640962.
- [29] Y.-S. Lee, Y.-Y. Fanjiang, C.-H. Hung, and Y.-S. Huang, "BLE signal processing and machine learning for indoor behavior classification," *Sensors* 2025, vol. 25, no. 14, Jul. 2025, doi: 10.3390/s25144496.
- [30] S. Natha, M. Leghari, M. A. Rajput, S. S. Zia, and J. Shabir, "A systematic review of anomaly detection using machine and deep learning techniques," *Quaid-e-Awam University Research Journal of Engineering Science and Technology*, vol. 20, no. 1, pp. 83–94, 2022, doi: 10.52584/qrj2001.11.
- [31] R. Sanchez-Iborra, A. Zoubir, A. Hamdouchi, A. Idri, and A. Skarmeta, "Intelligent and efficient IoT through the cooperation of TinyML and edge computing," *informatica*, vol. 34, no. 1, pp. 147–168, 2023, doi: 10.15388/22-infor505.
- [32] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," *IEEE Sens. J.*, vol. 22, no. 22, pp. 21362–21390, Nov. 2022, doi: 10.1109/jsen.2022.3210773.
- [33] J. Kwon and D. Park, "Hardware/software co-design for tinyml voice-recognition application on resource frugal Edge Devices," *Applied Sciences*, vol. 11, no. 22, p. 11073, Nov. 2021, doi: 10.3390/app112211073.
- [34] D. L. Dutta and S. Bharali, "TinyML Meets IoT: A comprehensive survey," *Internet of Things*, vol. 16, p. 100461, Dec. 2021, doi: 10.1016/j.iot.2021.100461.
- [35] T. Yang, A. Cabani, and H. Chafouk, "A Survey of recent indoor localization scenarios and methodologies," *Sensors* 2021, vol. 21, no. 23, Dec. 2021, doi: 10.3390/S21238086.
- [36] P. Spachos and K. N. Plataniotis, "BLE Beacons for indoor positioning at an Interactive IoT-Based smart museum," *IEEE Syst. J.*, vol. 14, no. 3, pp. 3483–3493, Feb. 2020, doi: 10.1109/jsyst.2020.2969088.
- [37] V. Bloch and M. Pastell, "Monitoring of cow location in a barn by an open-source, low-cost, low-energy bluetooth tag system," *Sensors* 2020, vol. 20, no. 14, pp. 1–20, Jul. 2020, doi: 10.3390/s20143841.

- [38] D. R. Farine, J. Penndorf, S. Bolcato, B. Nyaguthii, and L. M. Aplin, “Low-cost animal tracking using bluetooth low energy beacons on a crowd-sourced network,” *Methods Ecol. Evol.*, vol. 15, no. 12, pp. 2247–2261, Oct. 2024, doi: 10.1111/2041-210x.14433;
- [39] D. C. K. Yuen and B. A. MacDonald, “Vision-based localization algorithm based on landmark matching, triangulation, reconstruction, and comparison,” *IEEE Transactions on Robotics*, vol. 21, no. 2, pp. 217–226, Apr. 2005, doi: 10.1109/tro.2004.835452.
- [40] S. O. Ooko, M. M. Ogore, J. Nsenga, and M. Zennaro, “TinyML in Africa: opportunities and challenges,” in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2021, doi: 10.1109/gcwkshps52748.2021.9682107.